Designing a Web Application to Scrape Data From the Last.fm Platform

Independent Study Thesis

Presented in Partial Fulfillment of the Requirements for the Degree Bachelor of Arts in Computer Science in the Department of Mathematical & Computational Sciences at The College of Wooster

by
Andrew Fischer

The College of Wooster
2026

Advised by:

Drew Guarnera

THE COLLEGE OF
WOOSTER

Abstract

Last.fm is a website that tracks its users' music listening data, and is known for making that data available for developers to create informative web applications with it. This project seeks to create the Last.fm Popularity Calculator, an application joining this ecosystem, with a slight twist: obtaining certain data via web scraping that is unavailable through the site's official channels. The software seeks to accept a user's recent listening history and provide them with a breakdown of how popular the artists they listen to are.

This paper maps out and discusses the key conceptual subjects that go into the creation of the application: application programming interfaces (APIs), which connect discrete systems or internal components; web scraping and how to navigate its inherent legal and ethical challenges; software design and the architecture of different components in a single system; database systems and what type of system would be most useful to this project; and how to design and render informative data visualizations. The implementation of the software is then described. The frontend is able to accept an input file from the user and send it to the backend to handle the web scraping and processing of the collected data. Certain data is cached in a database, and the results are returned to the frontend to be rendered by the user's browser into a scatterplot.

This work is dedicated to the future generations of Wooster students.

Acknowledgments

First, I would like to thank my advisor Drew Guarnera and Junior IS professor Heather Guarnera, for helping me channel my obsession with this silly website into both substantive research and functioning software. Being able to envision, research, and build things based on whatever interests me was part of what drew me to computer science, and I'm excited to continue doing it after graduation.

Thanks also go to Will Sieber, whose suggestion last year to handle my LaTeX locally made all of the formatting a breeze, and to Tamir Hussein, who got me into music tracking in the first place.

Contents

List of Figures

CHAPTER 1

Introduction

Music has always been a social phenomenon, and that's never been more true than in the internet age, with discussion and discovery being easier than ever before. In the streaming era, the internet has become an important hub for both music listening and discovery, as services have been created not just for people to play music, but to do things like write reviews, catalog their collections, or chronicle their listening histories.

Larger web services like the sales platform Discogs or the listening tracker Last.fm make data from their platforms available to others, allowing an ecosystem of smaller web apps run by independent developers to proliferate using their data, which is for the most part freely accessible. This project will concern the Last.fm service and its associated web app ecosystem.

1.1 About Last.fm

Last.fm is a web service that collects music listening data through an API. Users connect their Last.fm account to a supported music player like iTunes or Spotify, which sends data about each song played through the software. This data is used to build a comprehensive record of everything they've listened to while connected. The site uses this data to create statistics, visuals, and tools with the data it receives, which will be expanded on throughout this section.

Every song played by a user creates an entry in Last.fm's database. Each contains a song title, artist name, and timestamp, with optional entries for an album name and album artist. They call these entries "scrobbles", after the name given to the software Last.fm runs on, the Audioscrobbler. An example of how these scrobbles are displayed on a user's profile is shown in Figure 1.1

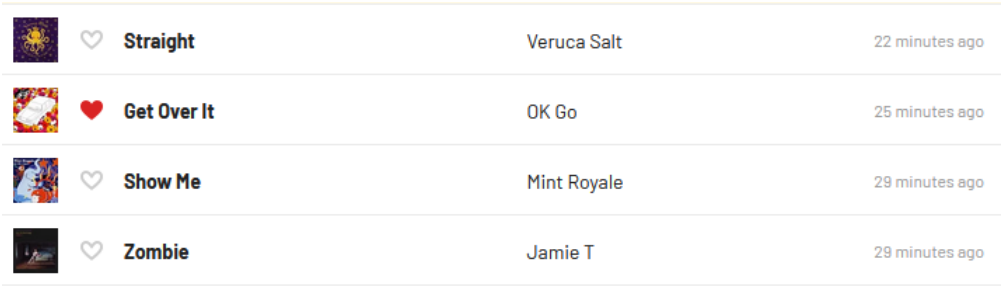


Figure 1.1: Four scrobbles on a user's profile, each track displaying their album, title, artist, and timestamp

Entries can be deleted by their user at any time, but can only be added within two weeks of the current data and time. This makes data from more than 14 days ago slightly more trustworthy, as new entries cannot be added after that time.

1.1.1 Artist profiles

While the main purpose of Last.fm is to build profiles of users' listening data, the site also builds profiles of each artist, album, and track that gets submitted. Each profile gets its own dedicated webpage that contains an editable wiki, a six-month history of how many unique listeners the artist/album/track had on each day (shown in Figure 1.2), and other user-generated widgets like a schedule of live performances for artists and a gallery of cover art for albums. These pages are auto-generated the first time an artist/album/track is submitted by a user and automatically refresh their information daily

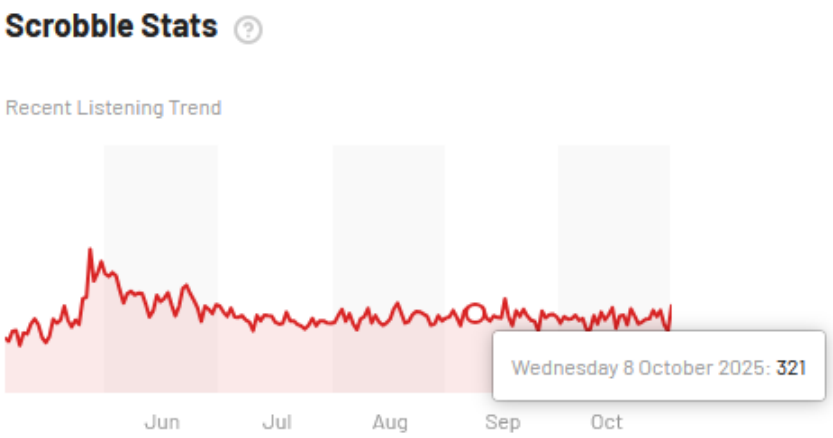


Figure 1.2: An example of Last.fm's graph of artist's six-month listening history

1.1.2 Last.fm's "Mainstream Score"

Last.fm delivers listening reports to its users every week describing their listening trends over that period of time. While all users receive these reports, paying users get access to a few extra stats and charts, including the Mainstream Score, a metric telling users how popular the artists they've been listening to are. The tool uses a largely unspecified method to assign a popularity weight to each artist, and provides the user a normalized score out of 100 and the identity of the most obscure artist they played in that time.

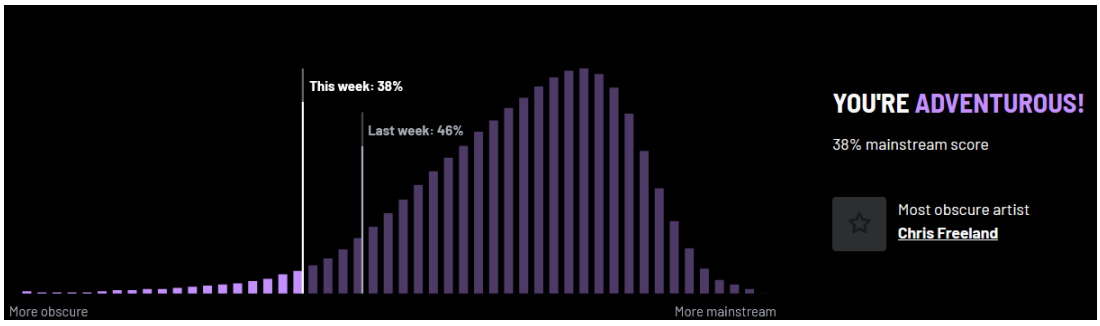


Figure 1.3: An example of how Last.fm represents its Mainstream Score

While this feature provides interesting information to users, it has a few key weaknesses. First, it is only available for paying users, those having a Pro subscription to the site. Second, it offers no further information than the two basic points of information

shown in Figure 1.3 – even the heights of the bars of the histogram are hard-coded and do not change from week to week. Data about the popularity of each individual artist is being collected by the site, but is not available to users in any way.

The aim of this project is to create a web app that uses scraping – a process to remotely obtain information from webpages – to collect publicly available data on the popularity of certain artists and compare it with listening data supplied by a given user. This will improve on the Mainstream Score in two key ways: first, it will be free and open-source, with the source code uploaded to GitHub for anyone to view or modify as they see fit; second, it provides more information to the user than Last.fm provides, ideally showing the user which artists contributed most to the weighting of their score. The site will take in as input the number of times a user has listened to different artists in a chosen timeframe and will output data about how mainstream the music taste represented by the input data is. A goal of the project is to fit into the wider ecosystem of Last.fm web apps while being the first of its kind to integrate input data from both scraping.

This paper will be laid out as follows: chapter 2 will cover background information on the relevant topics of web scraping and Web APIs, chapter 3 discusses the standards and practices of how web apps are designed and deployed, chapter 4 discusses the design and structure of databases and how web apps use them for persistent data storage, chapter 5 discusses the data visualization principles behind the visualizations the project will use and the nature of the tools used in designing charts and graphics for web apps, chapter 6 discusses my implementation of the software using the tools and concepts laid out in previous chapters, and chapter 7 discusses future plans for adding further functionality to the final software. Finally, chapter 8 concludes the paper.

CHAPTER 2

Background

This section will discuss multiple topics pertaining to the creation of a web scraping-focused application. First discussed will be the Application Programming Interface (API), a concept crucial to the scope of the project and important also to web and software development as a whole. Next discussed will be the concept of web scraping, including the mechanisms used to perform it, an example of the concept in action, and potential ethical considerations.

2.1 APIs

An Application Programming Interface (API) is a connection between two computer systems, as opposed to a connection between a system and a user. The term can be used to describe both how a software makes its output available to external programs and the ways in which internal software components make information and outputs available to other components. In the context of web and internet services, web APIs accept, handle, or distribute the content of a website or web app. Unlike other, internal APIs, web APIs are usually designed to be exposed over a network and used remotely by multiple users [5].

2.1.1 Purposes of Web APIs

In most traditional web pages, any information the site wants conveyed to the user must be taken from the original code and rendered graphically into a form the user can directly view. As web APIs represent computer-to-computer connections, there is no need to create visual elements and data structures that useful to human users. The two main benefits this creates are of automation and accessibility. Without the overhead of rendering graphical elements and with the ease of querying a system programmatically (i.e. via code), receiving and executing requests becomes a more efficient process, making it easier for it to be automated. The availability of data on-demand through an API makes it possible for other sites or services to make of its data, significantly increasing its potential reach and usefulness.

2.1.2 HTTP and RESTful APIs

Web APIs often communicate, like traditional webpages, via the Hypertext Transfer Protocol (HTTP), which sets an established protocol for communication over the web. HTTP facilitates a simple system of requests and responses between clients and servers, respectively, rather than a constant stream of data between the two. Requests and responses must each have a particular structure to comply with the protocol. Requests contain the following:

- An HTTP method, most commonly the verbs GET, POST, PUT, or DELETE (which tell the server to fetch, add, update, and remove content, respectively). Other verbs and select nouns like OPTIONS (which asks for specifications to make valid requests to a resource) or HEAD (which functions as a GET request that ignores any returned body content) also exist.
- The path of the resource to fetch; usually taking the form of a URL section like /api/v1/customers

- The version number of the HTTP protocol being used
- Optional headers – informational metadata fields – that may or may not be required by the server to convey additional information
- A body containing the content of a request for certain methods like POST

Responses similarly contain a mix of required and optional components that form a consistent structure. They contain the following:

- A status code, indicating if the request was successful – if so, what was accomplished and if not, why not
- A status message, a non-authoritative short description of the status code
- The version of the HTTP protocol they follow
- Optional HTTP headers, like those for requests
- A body containing the content of a response for methods like GET that ask for resources

REST (Representational State Transfer) is a style of API design to make systems that communicate over HTTP using simple, predictable operations. REST-compliant (usually referred to as "RESTful") APIs model resources like files as identifiable URLs, and make all interactions with those resources stateless, meaning each request contains all the information the server needs to process it [15]. A hallmark of REST is closely adhering to the standards of HTTP (e.g. organizing request routes correctly among the GET, POST, PUT, and DELETE methods, despite doing so being arbitrary and optional). By following clear conventions and leveraging HTTP's existing infrastructure, REST makes APIs easier to understand, integrate with, and scale.

2.1.3 Other API structures

Though the structure is extremely popular and often-used, REST is not the only structural option for designing an API. Other architectures exist to cover different use cases and different methods of sending and receiving information between client and server entities.

2.1.3.1 HTTP Server-Sent Events

Server-sent events (SSE) involve a client listening for data from a server and often involve a consistent stream of information. It is, in part, a reversal of the traditional REST structure, as the client waits for a server to send a specific form of information rather than the other way around. This architecture is useful when data is updated often enough that making individual requests would become overly-resource-intensive. In practice, a client sends a message to a server indicating that it is ready to receive events. Events are then sent from the server to the client at the frequency the server chooses. If and when the stream has to end, the server must communicate this fact to any clients receiving messages from it. In some systems a client may send a communication to the server that indicates the server's task is finished and to cease function, but the final signal that a stream is closed must come from the server to adequately notify the clients connected to it [19].

Some examples of SSE are often seen in public in the form of informational tickers – scrolling lines of text dispensing a slow, constant stream of information. These tickers are usually displaying live, up-to-the-minute readouts on topics like stock prices, sports scores, and breaking news – some of which change at regular intervals and some of which do not. Tickers are an obvious "real-world" example, but SSE-based connections need not be seen directly by humans at all. Other software can be listening for events and take action beyond simply displaying the information to a user. Think a financial group creating a program to take a certain action when the price of a volatile product or stock behaves in a certain way. SSE optimizes this style of communication by clearly

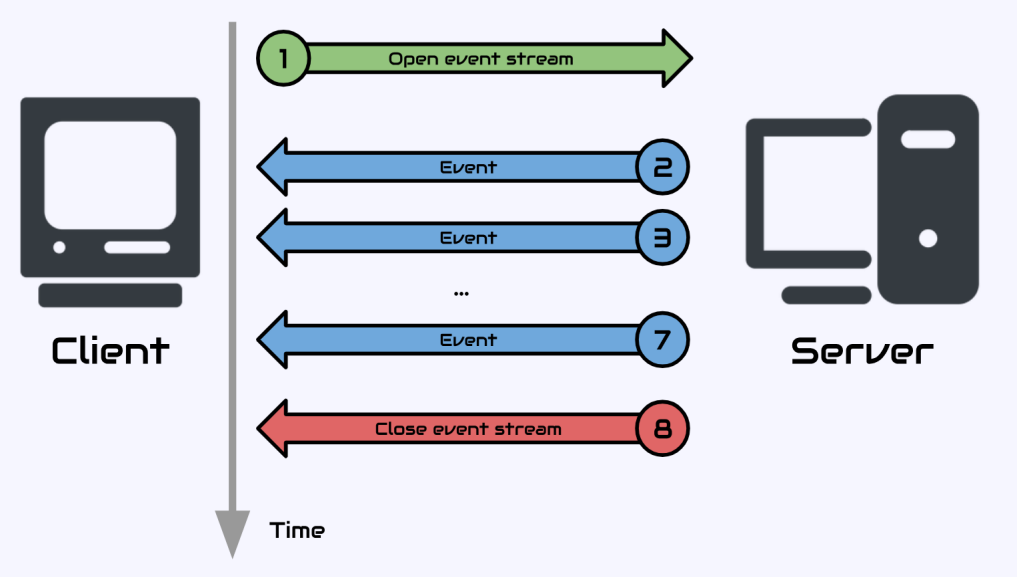


Figure 2.1: A diagram of the life cycle of a Server-Side Events process

delineating the different purposes of the client and server, allowing them to perform no more than their basic, necessary functions.

2.1.3.2 Event-driven architecture

Event-driven architecture (EDA) refers to APIs where the server (or other event provider) sends out messages or signals for clients to receive on open channels. It differs from SSE in that the clients do not typically interact with the server and do not need to notify servers of their presence – instead, the server fulfills its function by sending out events regardless of which specific clients may or may not be listening. This simplifies the role of the client, as its only role is to 'tune in' to events from the server at its leisure. This contrasts with SSE, where the client must maintain an active role in at least notifying the server of its presence. This structure is often used on internal APIs, meaning communication that occurs within a single piece or package of software rather than across multiple or over the internet as a web API. Depending on whether an EDA system is internal or not affects the specific methods with which it connects its clients to its servers.

Focusing on the use of EDA in an internal manner within a single, enclosed piece

of software, a separate component is needed between the client and server components to facilitate their communication. In this context, such a component is known as a message broker. Message brokers facilitate communication between multiple systems regardless of the implementation of each system or the communication protocol they use. Information is received, translated if necessary, and routed to the appropriate destination. Naturally, these brokers are not a concept unique to the EDA model – they are rather a type of middleware that exists to couple multiple discrete systems together (the concept of middleware and its role in web development is explored further in chapter 3). Message brokers are nonetheless a more important aspect of EDA because the server's lack of knowledge as to where the messages need to be physically sent often requires an intermediary to perform this routing.

There are two different broad styles message brokers use to facilitate and handle communications. The one used by EDA systems is "Publish/subscribe messaging", where servers send messages to the broker with organizational labels such as tags or categories, and clients listen for messages that match whatever combination of labels (if any) they wish to receive [9]. An example of a broker operating in this style is shown in Figure 2.2.

The other style a message broker can use is "Point-to-point messaging", where each server message is intended for a specific client, and is to be consumed only once. The broker's function in this style is to route (and possibly translate) the message from the server to the correct client, whether the routing information is given beforehand by the server, comes from the client, or is stored or generated elsewhere.

2.1.3.3 WebSockets

WebSockets is another style of API that, like SSE, allows for a constant stream of data. The key difference is that WebSockets allows that data to flow bidirectionally, meaning textual and binary data are able to be sent and received by the client and server at all

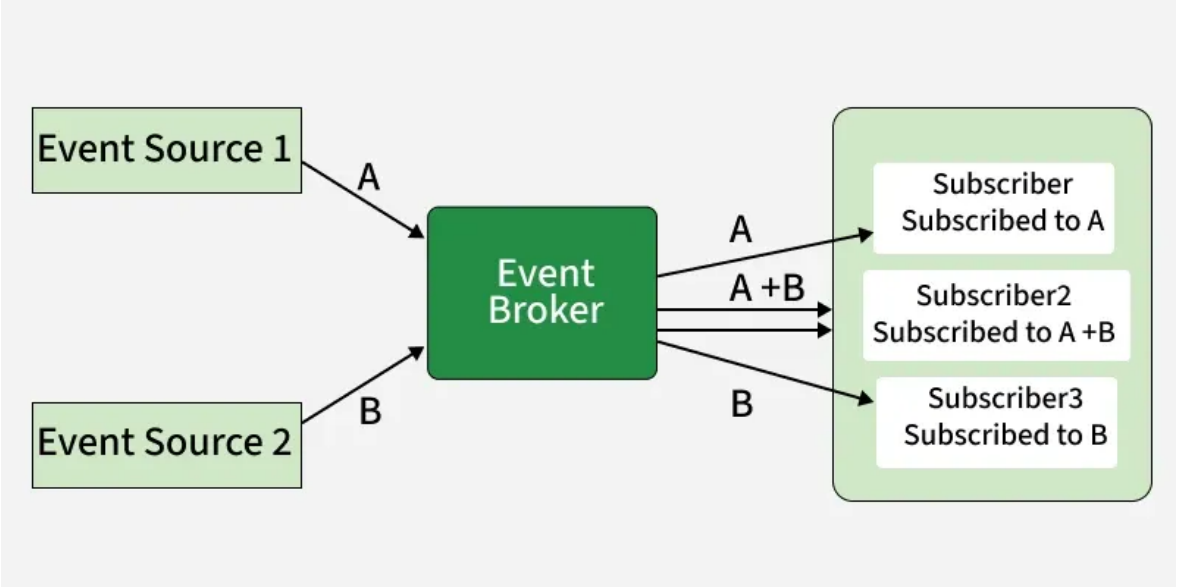


Figure 2.2: A diagram of the role of an event broker in communication between software components using the Publish/subscribe model

times as long as the connection is open. This method can become very resource-intensive, so care must be taken in what information is allowed to be sent over the connection.

Most use cases for WebSockets involve simple text-based data that needs to be updated very frequently. Examples include ride-share apps that let drivers and riders track each other's location for a rendezvous, instant messaging services that notify recipients immediately when new messages are received, and multiplayer video games, where players' inputs must be tracked and managed while being simultaneously integrated with other players in the same virtual environment.

Figure 2.3 represents an example of the start-to-finish process of a WebSockets connection. Because the client and server are both equally responsible for the sending and receiving of data (though one might have a greater responsibility than the other), they must both apprise the other of their respective presence and readiness to open a connection. Similarly, whichever side that is responsible for the cessation of the connection must notify the other so the two sides can take the appropriate measures.

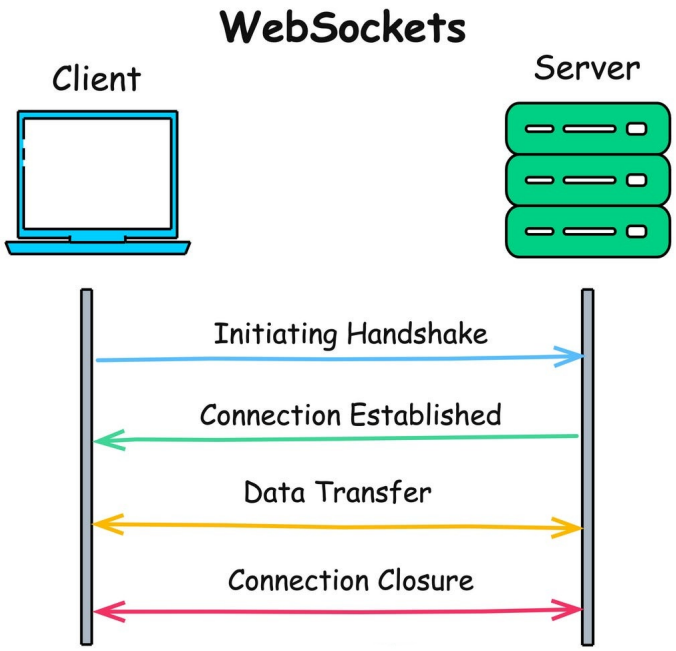


Figure 2.3: A diagram of a sustained bidirectional connection using WebSockets

2.1.4 The Last.fm API and its Limitations

In addition to receiving listening data from users, Last.fm maintains an API that allows web developers to use some of the data Last.fm collects and generates to create their own web tools using listener data. These tools often take the form of websites that create more detailed visuals of users' data than the official site provides. However, there is an important limitation: not all data shown on the site is available though the API. Notably, the six-month listening history for each artist/album/song is absent from the API's methods.

One popular web tool using the Last.fm API is the Last.fm Mainstream Factor – a site that uses API data to accomplish the same goal of this project: to create a simple numerical score to represent how mainstream a given user's listening history is. However, without access to the six-month daily listener history, the only other data available through the

API that can be used to represent an artist's popularity is their amount of all-time listeners – a count of every unique user who has listened to that artist at some point.

The site works by getting each artist's all-time listeners, converting it to a percentage of the largest listener base of any artist (currently Coldplay), and then averaging the artist's percentage for each track they played in a given timeframe. For example, Doechii has about 15% the amount of listeners as Coldplay, so if you played her songs nine times and a Coldplay song once, you'd end up with a score of 23%, with the larger share of Doechii songs weighing the average toward the bottom end of the scale.

This is a poor method of approximating an artist's popularity, especially over spans of time measuring one year or shorter – the maximum timeframe the Mainstream Factor can measure. It overemphasizes longer-standing artists and one-hit wonders who have many users listen to them at least once, but far fewer that listen regularly. It is also unable to take into account short-term effects on listenership such as album releases, where the amount of users listening to an artist will change dramatically over a course of weeks or even days.

To create a better program to determine how mainstream a user's listening history is, we would ideally use the data from the six-month listener history graph contained on their profile page. Since the API does not offer this information, an alternate method of gathering this data must be used. This is where web scraping comes in.

2.2 Web scraping

Web scraping is the process of obtaining the source code of webpages to collect specific data from them, usually using automated software. Whereas an API provides a direct portal where users can request data from a site's backend, web scrapers request whole pages just as a web browser would, but extract the targeted data from the source code rather than displaying it.

Webpages exist in HyperText Markup Language (HTML), which contains the structure and content of a webpage. When your browser loads a page, it sends an HTTP request to the web address specified by the URL. The site's server then sends the HTML located at that address to the browser, which renders it to be visible to the viewer. Web scrapers also request a site's code, but instead of rendering it, they keep it in text form and extract what they want from it, be that a data table, word count, list of external links, or whatever a user may want to know [17]. Though other functions like client-side JavaScript provide additional features and interactivity for most pages, for most sites the HTML accessed by a browser or scraper will contain the bulk of a page's content.

2.2.1 Mechanisms

The structured nature of HTML files is very useful to scrapers. With HTML, pages are organized into a hierarchical structure, with elements like paragraphs, lists, data tables and embedded pictures and video contained in marked sections for he browser to properly render them. The classes and tags added to HTML elements for identification by associated style sheet files also come in handy, as a scraper can read these groupings just as easily as a style sheet can. For example, if a sample of text is marked ..., the scraper will be able to identify all highlighted text on a page and separate it from the rest.

An important permutation of this concept is the ability to find the links that a page contains, which are also specially marked in HTML. Keeping a running list of links allows scrapers to traverse to those pages next, essentially traveling on a connected graph of webpages. Programs that perform this function are called "web crawlers" and are used to build sitemaps, collect data from many pages, or simply store all the code it finds to archive the state of each page at the time of retrieval.

2.2.2 An example scrape: robertchristgau.com

Figure 2.4 shows the HTML of a simple webpage in which all of the content is arranged in HTML's <table> format, which consists of <tbody>, <tr> (table row), and <td> (table data) tags. The figure shows the structure of the outer table, extending to three <td> tags. In the case of this webpage – an archive of music critic Robert Christgau's album reviews for a particular artist, The Roots – the first two <td> tags contain the page's header and sidebar (each formatted using another nested table, amusingly), while the third contains the reviews, contained in another structure shown in Figure 2.5.

Figure 2.4: An example website's HTML structure

Unlike the other two outer <td> tags, this section does not encapsulate its data in a <table> tag. After a few <h2> (heading) and <ul> (non-enumerated list) tags to display the page info, the page separates each entry into <p> (paragraph) tags, which by default display the text within each of them in the order they appear in the HTML document. The <p> tags contain more tags delineating the name of the album, the label and release

year, the text of the review, and the rating (two asterisks for the displayed entry, indicating an above-average grade on Christgau's esoteric scale). Figure 2.5 illustrates this structure.

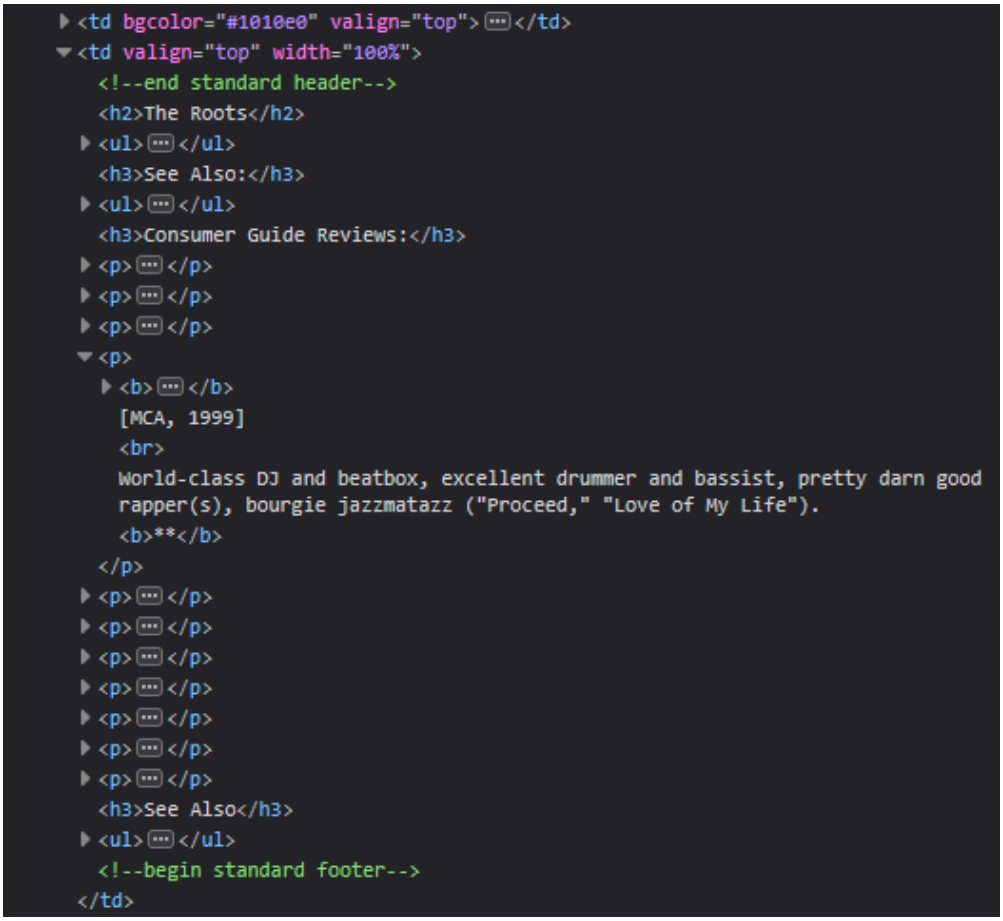


Figure 2.5: Structured data within a webpage

On this website, each page of reviews is structured in the same way, with three tables, the third one containing the page's main content. This gives a scraper some flexibility in how it would search for and access any information it wanted: it could find all the tables in the document and find the content table by looking for a table that contains the specific combination of header and paragraph tags, or simply count to the third table on the page and access it, as all pages' third tables will be the content tables. This example shows how the structure of a simple HTML file is seen from the perspective of a scraper.

2.2.3 Conceptual limitations

If a scraper is designed to retrieve specific information from a webpage it visits, it often depends on the structure of the HTML to be able to find it quickly and easily. If a site's format and page structure is inconsistent across multiple pages, or if it changes altogether, the scraper will break. Clever use of error handling or fallback conditions can mitigate these issues, but scrapers are, by nature, largely at the mercy of whatever site they are accessing.

Because web scraping accesses webpages using the same channels a normal user opening a webpage would, some discrepancies between the two systems may cause problems. Unlike web APIs, which are designed to be accessed programmatically and not directly by users, scraping automates and streamlines interactions with a system designed for human interaction. This could lead to a scraper being identified as a malicious actor (falsely or not), and introduce ethical challenges detailed in the next section.

2.2.4 Legal and ethical considerations

Because websites are typically designed for users to access and view them in a web browser, two main ethical concerns can arise from loading large amounts of web pages to gather data:

First, there are concerns of server load. Scrapers can request webpages far faster than humans can usually read them, so requesting many pages from the same site will lead to your scraper demanding an outsized share of the processing power a site can provide. Second, some site owners might not want data contained on their site to be collected and stored, even if on the surface it is accessible. In the early days of web development, these concerns led to the creation of a web design standard: robots.txt [17].

robots.txt is a simple text file that most websites will store in their root directory (i.e., the robots.txt file for example.com could be found at example.com/robots.txt). Its

purpose is to tell web scrapers (and other autonomous agents outside the scope of this paper) which pages on the site, if any, they are allowed to access. Some sites don't care and leave the file empty, some are very restrictive and only allow access to search engines like Google, others operate somewhere in between. In legal cases involving unwanted web scraping, courts have generally interpreted the lack of a restriction in a site's robots.txt file as consent for a web scraper to access that part of the site [17]. While workarounds certainly exist, many popular consumer web scraping tools will not allow you to access sites restricted by a robots.txt file.

Last.fm's robots.txt file currently prohibits scrapers from accessing users' listening history and data for their profile pages, but does *not* prohibit access to the artist profile pages where the charts we want to access are stored. This means that the project is in the clear functionally, ethically, and legally until Last.fm changes their terms of service or their robots.txt file to explicitly prohibit its actions.

CHAPTER 3

Full stack web design

This section will introduce the concept of the tech stack and discuss concepts of full-stack web development and their relevance to this project's goals. Starting with a primer on the basics of web applications and information transfer over the internet, following with sections on conceptual differences between single- and multi-page applications, concepts and examples in the software architecture of web apps, continuing with an overview of the deployment process and the role of containerization, and concluding with an assessment of the process behind constructing a tech stack for this project.

3.1 The Tech Stack in Web Development

In the field of computer programming, a software stack is the set of software components required to run and support an application [2]. The stacks one uses for different applications can vary, but popular combinations of components have proliferated over time due to the compatibility of certain components with one another. In web development, a common classification of stack components is labeling tools as part of the "frontend", which encompasses the presentation and organization of a site's content, determining how a site is made to appear to a user; or the "backend", which encompasses the logic of the site, processing user requests by routing them to the right component, storing data necessary to the function of the software, and managing

application operations. Some also use "middleware" as a separate term, covering steps like authentication, the routing of information between components, and liaising with third-party services. Going forward, this paper will use the term "middleware" in this way.

An example of a popular web development stack is MEAN, an acronym standing for the four components it contains:

- **MongoDB**, A database management system to store information needed by the application. It is non-relational, a concept explored in chapter 4
- **Express.js**, a framework for designing web applications with JavaScript
- **Angular**, a framework for designing user-interfaces
- **Node.js**, a runtime environment allowing JavaScript to be executed on a server, outside of a browser.

Each component in the stack fulfills a different purpose, and together are able to run a web application. Node.js and MongoDB make up the backend, performing the processing of information and storing data, respectively. Angular handles the frontend, providing the tools to build a flexible user-interface. Finally, Express.js is the middleware – it routes code and information between the other modules. Since MEAN does not require a specific operating system to run within [20], it is known as an "OS-agnostic" stack. An example of the opposite – an "OS-level" stack – is LAMP, which uses the Linux operating system. The rest of the LAMP acronym stands for Apache, the framework to run the backend server; MySQL, the database manager; and Perl, as the primary coding language used.

Even when part of the same unifying software, different stack components must still have an established method of communicating with one another. As discussed previously in chapter 2, components communicate with one another through APIs. HTTP (usually

through a RESTful API) forms the boundary between the frontend and backend, while middleware and backend components have their own methods of communicating with one another. Examples of how multiple components can be organized within a system will be discussed later in the chapter.

There is no set framework for what kinds of component constitute a specific "stack"; rather, named stacks like LAMP and MEAN highlight the most important and relevant components. Another factor is that certain tools may overlap with the functionality of others (for example, one backend framework may include functionality to communicate directly with a database while another may need an intermediary), meaning a rigid categorization of components would not be helpful. Types of component that exist in one stack might be redundant or unnecessary in another.

3.2 Single page vs. Multipage

One key design decision crucial in the early stages of creating web applications is the choice between creating a single-page or multi-page application. The distinction is fairly intuitive: single-page applications load only one HTML page initially and use client-side code like JavaScript to perform dynamic functionality such as fetching data and changing the user-interface. Multi-page applications, on the other hand, consist of more than one HTML page [2]. While the actual size of the sitemap in these applications can vary widely – anywhere from a small handful of pages to dozens – it is seen as a significant distinction to go from one page to multiple, no matter the exact level of the difference.

The key difference marking the distinction between the two types is that in single-page applications, when a user first lands on it, the entire frontend of the application is downloaded by their browser. Even if the application has multiple pages (which, while profoundly unintuitive, does not preclude it from being called "single-page"), the content of each page is sent from the server to the client in one response, and any page-to-page

routing is handled by the application's client-side code that the user's browser executes. If, through the user's use of the application, more information is needed from the server, an API call will be made to request it. But the overall distinction between single-page and multi-page is how the application's content is delivered to the user.

Single-page applications are a relatively new concept in the history of the web. In the early days of the internet, webpages would each be sent from server to client in discrete requests, much like a multi-page application today. Over time, as personal computers and web browsers became more advanced, the capability of users' devices to provide interaction and functionality on the client-side increased through tools such as JavaScript. Over time, client-side functionality advanced to the point where for certain web tools it made more sense to send a website's entire functionality to the user all at once and have it act more like a desktop application than a traditional website. Most modern websites use some level of client-side functionality, whether they're single- or multi-page, but a trade-off between the two still exists: for some sites it's beneficial to send the user a full application all at once, and for others it would be more efficient to separate different sections of the site's functionality and only need to deliver the content the user actually needs.

The decision whether to structure an application as single-page or multi-page depends on how a designer wants information to be organized. To give an example of each, we can look at Amazon and Google Maps. Amazon is a multi-page application where information about each product is contained on its own page. Because its catalog of items is so vast, it makes sense to cordon off each individual item into its own area with relevant information like price, availability, and user reviews. Other areas of the site like the search results page and a user's shopping cart are also their own pages, only being fetched and delivered to the user if they specifically ask (although a smaller sidebar displaying items in a user's cart is shown while browsing, the cart page contains more details not included on it).

Google Maps, on the other hand, mostly operates on one homepage, displaying a basemap of the globe, focused in on a user's location. The site's main features are displaying information about a selected location and mapping a route of transport between two places – both of which are accomplished without leaving the main page. Users select a place either by clicking or with the search bar. Information about the place is then displayed, along with options to give directions, save it to a list, etc. Despite its single-page nature, however, the entirety of information Google contains about world locations is – as one would expect – not saved into a user's browser as soon as they land on Google Maps. Rather, an API call is made when a user selects a location, requesting further information from Google's servers such as images and user reviews if the place is an established business. Despite needing additional information from the backend, the site incorporates the retrieved information into the existing page rather than navigating to a separate page for each location as Amazon does with its products.

3.3 Architecture

The tech stacks in modern software and development often comprise multiple components that interact to perform its functionality. As we saw earlier, different stacks can contain or emphasize different types of components while being agnostic on others. The structure of these components and how they connect with one another can be used as a basic definition of software architecture. Depending on the needs of the software for its stakeholders, it may need a different combination of components, or commonly-used components assembled and connected in a different way [6]. In web development, while the frontend and backend are commonly-used categorizations of services and components, there are many ways to structure an application, some blurring the lines between the categories.

This section will cover both high-level and low-level architectural concepts with a primer on the basics of web applications and the transfer of information over the internet.

3.3.1 Architecture styles

An architecture style represents a combination of design decisions and principles to organize an application in a certain way. It is the highest level of organization, not used to solve a specific problem but to enhance certain good practices in a particular type of system [6].

3.3.1.1 Monolithic Architecture

Monolithic architecture is a software development model where all components are interdependent and interconnected; every component is required to run the application. It treats the entire code as a single program, building a new project, applying frameworks, scripts, and templates, which makes testing easier.

The main drawback to this style of architecture is that it becomes difficult to manage or update the code base as it grows. As each element is interdependent, scaling the application can be complex. Introducing even a single point of failure can break the application.

Monolithic architecture is best used for simple applications that have any future scalability planned out in advance. Knowing as much about the future scope of a project's features in advance as possible helps determine what will be feasible or not under a given structure [22].

3.3.1.2 Microservices Architecture

Microservices architecture serves as the logical counterpoint to a monolithic environment. Using it, code is organized among independent services that don't rely on one another to perform their specific function and often communicate via RESTful APIs. Each microservice contains its own database and operates under its own business logic. Because it's loosely coupled (meaning components do not have to change significantly if another one within the overall software does), microservice architecture makes each individual

service much easier to add features to while maintaining continuous delivery. Developers can thus quickly adapt the software to changing circumstances.

However, maintaining a piece of software with multiple services comes with drawbacks. When the number of services grows, the complexity of managing them grows, too. Microservices architecture is a good choice for highly complex applications with the resources to maintain and scale multiple independent services [22].

3.3.2 Architecture patterns

In contrast to styles, architectural patterns are decisions and principles intended to avoid specific issues with the low-level implementation of the software. Essentially, the style comes first and determines the project's broadest level of structure, and the patterns determine the specifics of how each of the components interact with each other [6]. Below are the most prominent architectural styles and how each covers different possible use cases with their strengths and weaknesses.

3.3.2.1 Preface: The Client-Server model

The Client-Server model delineates two types of entity: clients that seek information and send structured requests for it, and servers that carry out these requests by fetching and processing the information, thus serving the clients. Often communicating through APIs, it is the server's job to stay open and available for requests, and the client's job to behave in a manner the server expects of them to best facilitate the transfer of information and handle the presentation of data, completing the path of information between the server and the end user.

Though technically a form or pattern of architecture, the idea of the client-server model is broad enough that it encompasses all web development to some extent – for a user to get any information from the internet, there must be some sort of server they get it from. Thus, the model acts as a kind of superset comprised by the different patterns detailed

below – each is structured around clients and servers in some way, but to different extents and sometimes with very different strategies.

The model also appears throughout the field of software development at large, including other subjects pertaining to web development. In chapter 4, we will discuss how certain database systems create their own client-server interface within a web server to facilitate the movement of information through the system before it is delivered to the web client. the following architectural patterns in this section are not so broad and will pertain for more directly to web architecture.

3.3.2.2 Layered

The layered pattern is a simple way to organize services in a web application. In it, information simply flows through different levels of the software to and from the user and the system in sequence, interacting with the code at each level before being passed on to the next one. The most common permutation of this pattern comprises four layers:

- *presentation (or UI)*, consisting of everything the user sees and interacts with. Pages, menus, text, images, etc.
- *business*, in which the business logic of the application is defined and operated
- *persistence*, containing mapping functions and other operations
- *database*, where information is persistently stored

While the simplicity of this pattern makes it easy to understand and useful for lightweight, small-scale applications, the lack of flexibility brought on by sending information through many consecutive layers can be inefficient for large applications. Since the layers are interdependent, it also introduces many potential points of failure should one or more layers develop a fault.

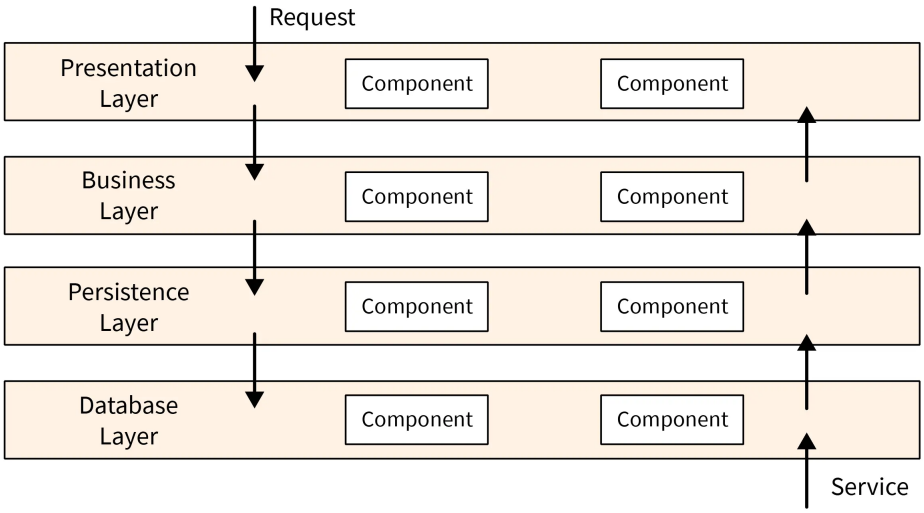


Figure 3.1: An example of layered architecture

Another important permutation of layered architecture is the **Model-View-Controller (MVC)** design. Put simply, the MVC pattern separates the components into three roles: backend logic, frontend presentation, and an intermediary to connect the two, respectively.

- **Model** – contains business logic, core functionality, and persistent data storage. It does not contain any logic that shapes how the data is presented to the user; that is the controller's job
- **View** – Displays information to and interacts with the user. Takes data from the Model and Controller, processes it, and dynamically renders it for a user.
- **Controller** – Processes the inputs the view receives from the user to send to the model and returns data to the view and the user from the model.

Broadly, it can be seen as organizing components into the frontend/backend/middleware trichotomy described earlier. This variation is less straightforward than some of the other architectural patterns, which can make it unsuitable for some projects that aren't prepared to handle its particular style of decoupling. It also illustrates how multiple functions can be performed by a single layer in such a way that the flow of data is not

interrupted. This manifests in how the controller handles the input/output of both the view and model, and how the model acts both processing and data storage. MVC doesn't necessarily differ from layered architecture, but rather separates the layers into three distinct groups. The concept is illustrated in Figure 3.2

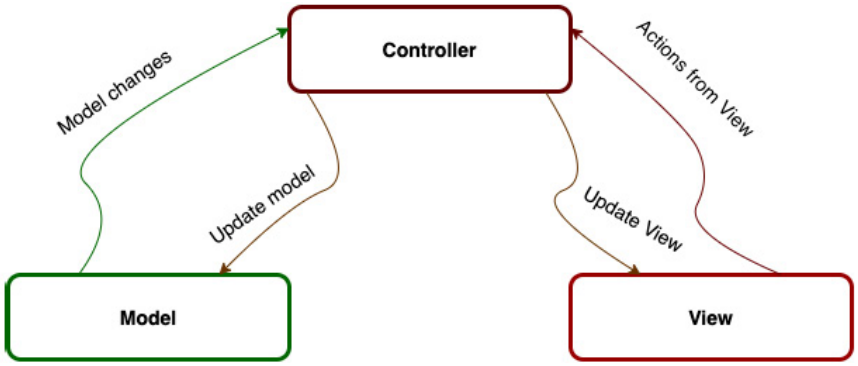


Figure 3.2: A simple diagram of the Model-View-Controller architectural pattern

3.3.2.3 Command and Query Responsibility Segregation (CQRS)

The signature aspect of the CQRS pattern is that it pares software down to just two main components: a user-interface and a data store. The user, through the UI, performs operations on the data store directly without having to go through an intermediary like the controller in the MVC pattern. In CQRS, data operations are separated into reading- and writing-based actions: queries and commands. Queries are used to gather data from the database storage; the user specifies what data they want and the application fetches it for them. Commands are responsible for changing data, and can carry out a wide range of actions on it. To actively modify, rather than just read, data from the store might be a more intensive task than the UI can perform on its own, which is where Command Handlers come in [6] – if needed, these serve as a single-purpose intermediary between the two main components and have the ability and authority to make changes to the data store.

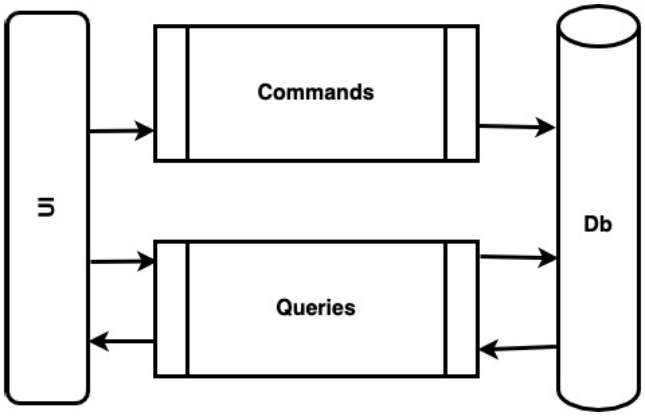


Figure 3.3: A simple diagram of the Command & Query Relational Segregation architectural pattern

Direct access to an application's data store can be a lot of freedom to give a user. Depending on the specific data storage component the application uses, certain restrictions can be placed on what type of actions users can perform. These restrictions can essentially function as a stripped-down form of business logic for the application without a separate component to handle it. Queries and commands can either succeed or fail, depending on their compatibility with the data store's restrictions, and send the result back to the UI, whether directly or through a Command Handler.

The advantage of CQRS that it separates the intermediate layer of the application into two wholly distinct components that can operate independently – this way more attention or resources can easily be given to one or the other if a particular project is more reading- or writing-based. It should not, however, be seen as a simple default for database-centric applications; if all you need in an application are the basic commands of creating, reading, uploading, or deleting data (collectively known as CRUD commands), CQRS is likely far more technically complex than is necessary. Usually working with GraphQL, CQRS is a pattern that is geared toward working with it and other query languages to fetch and manipulate data.

3.3.2.4 Microfrontends

Where the user-facing functionality of an application is contained in a single layer in many other patterns, microfrontends refer to applications that split the user interface (UI) into multiple services, usually worked on by different teams, each with different tech stacks. The pattern essentially extends the microservices style to the frontend, greatly enhancing the capability of the UI by breaking up the monolith. Instead of one single component being wholly responsible for rendering and managing the entire user interface, splitting this responsibility among microfrontends allows functions to be addressed by and linked between different components [10]. For example, a site that displays a list of options for the user to choose from like a restaurant might have one frontend handle the options, formatting them and how they're organized, while another handles background aspects of the page like the broad structure and how access to other parts of the site are displayed. An example of this is shown in Figure 3.4.

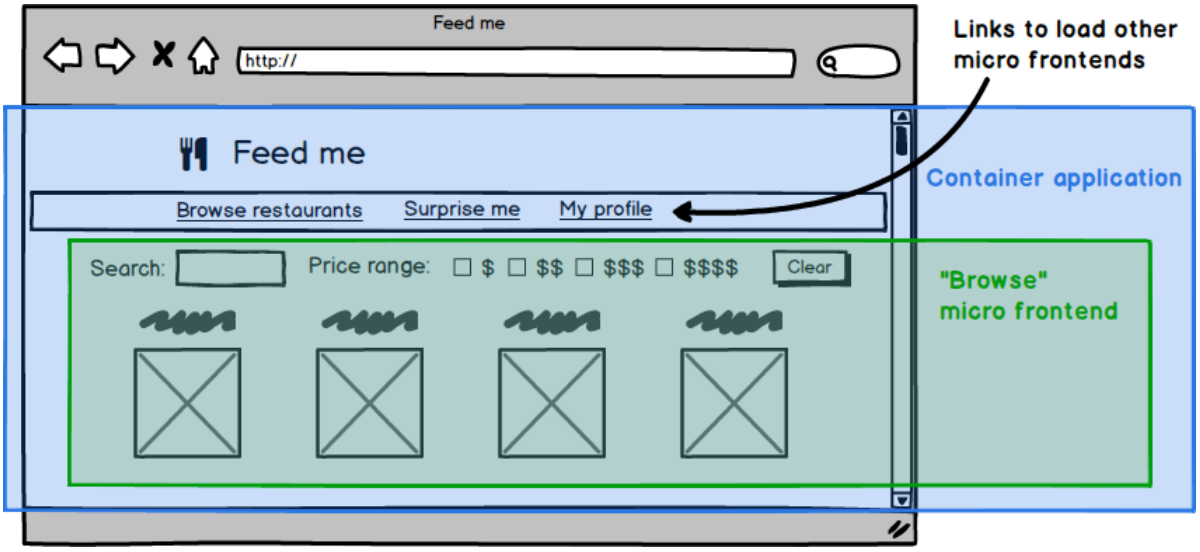


Figure 3.4: An example of a single webpage using microfrontends to manage separate sections of the site

The key advantage of microfrontends is the ability to break the development of each frontend away from the others, allowing different teams to work independently on

their designated frontend while coming into conflict with the others as little as possible. While the restaurant example is likely simple enough not to require multiple dedicated teams to maintain, a more complex application like a retail website or streaming service could conceivably have many teams with different priorities – some might be aiming to constantly innovate and change their area of the service to keep things fresh and up to date, while others may be content simply to make sure their frontends don't break or otherwise fall behind the progress of the rest of the service.

The main disadvantage of microfrontends is in line with the potential drawbacks of microservices in general: keeping multiple disparate components interconnected while maintaining a level of independence among them can easily be an extremely intensive task. Each component must maintain a relatively consistent system of inputs and outputs that let the others easily interact with. Testing is also a potential difficulty, as running tests on individual components may not capture issues that arise when connected to the larger whole. Thus, the level of independence each frontend can maintain is lessened the more they have to interact with each other to run tests on potential changes.

The emergence over time of the microfrontend pattern exemplifies the growing importance and complexity of user interfaces in the modern web. While naturally better-suited to large development teams and resource-intensive frontends, microfrontends serve as a proof-of-concept that it's just as possible to prioritize and expand the functionality of the user interface as with any other segment of an application.

3.3.2.5 JAMstack

Where microfrontends use multiple services to make the user interface more layered and complex, JAMstack (JAM stands for "Java, API, and Markup") moves in the opposite direction. The principal attribute of JAMstack is the lack of a traditional backend server – pages are instead sent to the user through a Content Delivery Network (CDN), which can send files to the user but does not hold data or process requests. To accommodate this,

rather than use dynamic webpages as all previous patterns generally do, JAMstack instead relies solely on static, pre-rendered webpages written in HTML and JavaScript. Without a web server, the client-side JavaScript code is responsible for handling communication with the CDN and any other microservices, such as data storage or identity management.

Unlike patterns focused on optimizing for the dynamism of their frontend and/or the scalability of their backend features, JAMstack optimizes for space and cost management by eschewing dynamic frontends and many common backend features entirely. This pattern is most useful for simple web tools and other applications without the need to increase in scale by adding new features over time.

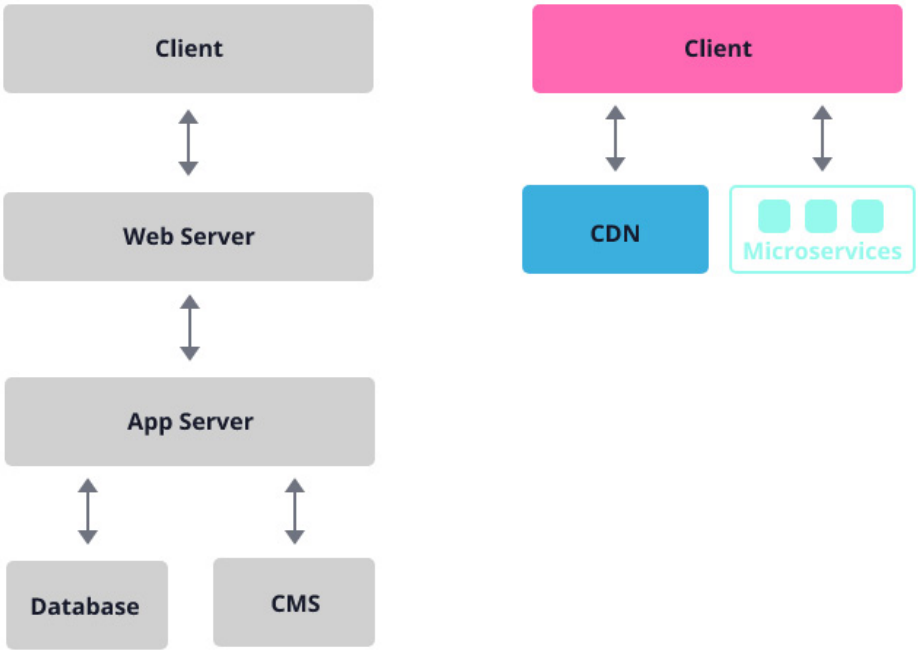


Figure 3.5: Contrasting the JAMstack (right) with a traditional layered architecture (left)

3.3.2.6 Flux

Flux is a pattern created by Facebook, designed to work with the service's React frontend components to build client-side web applications. It was created as an alternative to the Model-View-Controller pattern described earlier to avoid unnecessarily tangled

connections between the services in an application [18]. Flux has a unidirectional data flow, meaning data is always sent through the services in the same sequential order. In the most basic example, the app has three services that process an action taken by a user:

- **Dispatcher** – Receives the action taken by the user and determines both where to send it, and the order information is sent in
- **Store** – Contains the business logic, which it operates on data received from the dispatcher. Also manages the state of the application
- **View** – Receives specifications from the store and renders the information to be presented to the user. The view re-renders when the store announces changes and is responsible for creating new actions based on user input and interaction

Each of these services only sends data in one direction and receives it from the other without a need to double back to confirm or update data. This results in a simpler application that is easier to follow and debug while each service can be optimized for its narrow focus.

Flux was designed with a specific use case in mind: managing React components simply and efficiently. Its lack of two-way inter-service communication would not be a good fit for a project managing large amounts of information, but for a user interface-focused application that doesn't send data more complex than basic text and/or images, it makes perfect sense.

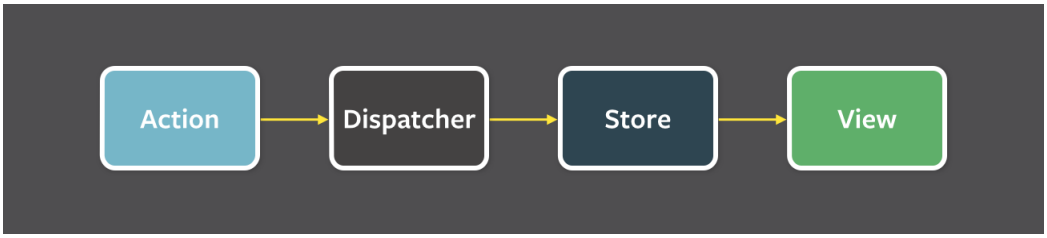


Figure 3.6: A simple diagram of the Flux architectural pattern

3.4 Containerization and deployment

The deployment of a web app is the process of transferring your software from the local machines used to develop and test it to a hosting provider that can make it accessible to end users. This usually involves paying for backend servers (though you can run them yourself) and buying a domain name to allow users to access your app as a website.

When applications move from the development to the deployment stage, the components of an application must have concrete ways of interacting with each other and the hardware (usually servers) required to become accessible to end users. A popular solution in recent times is containerization [2].

Containerization is a way of packaging an application together with everything it needs to run, like its code, system libraries, and configurations, into a single unit that can run on a physical or a virtual machine (VM). By existing at the OS level, containerization allows a developer to run multiple containers inside a single OS instance. While it reduces overhead and processing power, it also makes them much faster to start and more resource-efficient.

Tools like Docker build these containers from reproducible definitions (which they call Dockerfiles), ensuring the application runs the same way on different machines, maintaining consistency in development, testing, and deployment. Containerization tools allow teams to ship applications as portable, self-contained artifacts that can run reliably across different environments such as local servers or cloud platforms.

3.5 Building a tech stack for this project

To determine what elements of a potential tech stack would be most useful to this project, this section lays out the planned extent of its capabilities and discusses potential solutions and design choices pertaining to the architecture of the software as they relate to the concepts discussed within this chapter.

First, the user needs a way to communicate their Last.fm listening history to the application, either by uploading a file generated by a Last.fm export tool, or if the project is granted direct API access, entering their username for the application to collect the data on their profile. These initial tasks require very little of the frontend – either or both of a text field for a username and an upload window for a file. Once the user's listening history is obtained from either an input file or the site's API, it has to first go through some initial processing to generate a list of each artist they listened to, along with how many times each was played. Once these lists are generated, the most important process begins: the scraping.

Scraping is, comparatively, a much more intensive process than the other functions of this application. Consequently, the decision on how to undertake it will be important. While the idea of offloading the scraping of a listening history onto the user making the query would certainly save resources on the server side, it would not be easy or ideal – relying on the client's browser to do the work would necessitate running the code in JavaScript, which would deprive us of some of the key tools we've planned to use for scraping and data handling. In addition, the processing undertaken by the user's browser could last several minutes, depending on the size of their listening history; this isn't ideal, as keeping a single process running in one tab for so long would put an undue burden on the user to keep that tab loaded and running. The longer such a process would run, the greater the risk of a connection or other client-side error interrupting the process. Since running the scraping client-side has these potential drawbacks, the best solution seems to involve a more standard solution: running the processing and scraping on a web server.

Using a centralized server system for the scraping gives us significant freedom in choosing how to go about it. If data from the user is collected by a client-side UI and sent to a backend server, there are numerous ways that data could be processed and returned to the user. The processing would not be restricted to client-side tools and could thus freely use non-JavaScript coding languages, as well as libraries and modules that could

not be passed directly to a browser. In addition, the lack of dependence on the client-side after the initial upload means scraping and processing can potentially run without the user needing to keep their browser tab open, allowing for the possibility of implementing an extended queue of requests if users submit histories to the app faster than it can scrape the artists within them.

The primary function of the backend, once it has finished scraping the user's artists, is to perform data analysis to generate select insights, and synthesize the data into a form that can be displayed to the user. Since the use of a persistent data store is not required for these tasks, one is not strictly necessary for the project. There are, however, potential benefits an implemented data store can provide, which is discussed in chapter 4.

While we determined that the scraping and processing of data is best performed on the server-side, the next step requires consideration of whether the rendering of the final data visualization to be presented to the user should also be rendered server-side. While the specifics of the data visualization used in the project is discussed in detail in chapter 5, suffice to say that the data the application procures for visualization will not be particularly complex and can be displayed with fairly simple charting strategies. The question faced at this stage is: would it be more useful or efficient to generate the visualizations on the server-side before sending them to the client, or sending the data to the client-side for it to be generated by the user's browser?

Client-side tools like JavaScript libraries are much more prevalent for changing the visual appearance of a site than performing complex processes or calculations, so it stands to reason that generating visualizations and integrating them into the webpage would be more likely to be a task best operated on the client-side. Indeed, JavaScript libraries exist for transforming a dataset into detailed, even interactive, visualizations, which should be more than able to display the simple data from the server. With these facts considered, the best course of action is for the backend, once it's finished scraping and processing the data, to send it back to the frontend to be generated and rendered on the client-side.

In keeping with the general format of other Last.fm-based web apps and acknowledging the very specific task the application performs, it would be ideal to create it as a single-page application. For a user, the ideal experience with the site would be to upload their listening history, wait for the process to finish (if they want), and receive the results without having to navigate to a new page. If functionality is implemented to give the user a code to come back and view their results at a later date, it stands to reason that it should also be done in such a way so that they can access that from the main page, rather than a separate page for that purpose.

Taken all together, the flow if information through the application would go like this:

- The user loads the page and makes an upload to the frontend
- The frontend sends the file to the backend to extract the artists and perform the scraping
- Optionally, some data is cached in a data store (see chapter 4)
- The backend passes the processed data back to the frontend
- The data is rendered into a visualization via the UI

This is an example of layered architecture, where each component is used in sequence, and the pertinent data flows in both directions. Enough of the traditional tools like a frontend framework and a centralized server are needed that using a stripped-down architecture like JAMstack or CQRS wouldn't be feasible. For a small-scale data-driven web application, layered architecture is an ideal solution.

CHAPTER 4

Database theory and design

A database is often defined as an organized collection of information used to model some type of organizational process. In software, databases are a method of storing information persistently – meaning data is saved in a location that will not be lost after the program concludes or is refreshed [8]. In web development, databases are located on the backend and store information that is accessed by the server to process and send, in some form, to a client. This chapter covers various permutations and architectures of databases in the realm of web development.

4.1 Relational databases

The most common form of database is the relational database. Invented in and popularized since the late 1960s, these databases store information in "relations", which are commonly represented as tables representing a specific entity like customers or products in a retail setting [8]. Within each table, each row represents a unique permutation of that entity (e.g. a different customer), and each column defines a particular attribute of the record (e.g. a customer's name, age, etc.).

Every table includes a primary key: a field (or group of fields) in each table that makes each row uniquely identifiable within it. Relationships between tables are maintained with foreign keys, which associate one table's records to those in another via the first

table's primary key. Relational database systems use Structured Query Language (SQL) to operate on stored data. SQL is used to insert, update, delete, and retrieve information in a database. The relational model ensures data integrity, which makes it widely used in contexts where accuracy is essential. Important aspects of relational databases that serve to support data integrity and maintain transaction control are referred to under the acronym ACID:

- **Atomicity** means that transactions are 'all or nothing' – if a transaction cannot be completed in its entirety due to failure at any point, it is rolled back and not undertaken at all. Atomic transactions protect and secure data, but are computationally expensive and difficult to test.
- **Consistency** ensures that the state of a database is never captured in transition, and that database accuracy is never corrupted by an illegal transaction.
- **Isolation** means that transactions are executed statelessly, so concurrent transactions do not interfere with each other.
- **Durability** guarantees that successful transactions will remain saved within the database in the even of a system failure of crash.

ACID systems, as they are called, prioritize the consistency and integrity of data above all else and thus must take into account all possible sources of failure into account.

4.1.1 Components of web-based relational databases

Many web applications are data-driven – existing to store, generate, or otherwise process data for a user. There are several technologies and development strategies involved in integrating databases and database systems into web applications and their codebases.

A relational database management system (RDBMS) is software that persistently stores the contents of a relational database in a system and facilitates the management of its

contents and structure. Many RDBMS design their databases to interact with software through the client-server model in a manner similar to how users and web servers interact over the internet – the database functions as a server with its own address and ports that it uses to listen for commands from the client. However, not all database managers work in this way; ones that do not use a client-server model are discussed in section 4.3.

Like web servers, these database servers communicate with clients using an API. In its most basic form, a database's API lets it receive low-level SQL queries and return appropriate responses. These APIs are often separate from the RDBMS itself, but are tailored to each one (e.g. mysql-python and PyMySQL are APIs that work with MySQL) [2].

Some applications use an intermediary software or library to facilitate interaction with the database and its API called an object-relational mapper (ORM). An ORM works by representing aspects of a database in the style of an object-oriented programming language, and translating operations on such objects into commands the database recognizes. For example, an ORM like SQLalchemy works with Python code and uses operations it calls "read", "insert", "update", and "delete", among many others. These commands are not SQL themselves and would not be understood by a database's API, but are interpreted and transformed by SQLalchemy into a form the database management system and its API can understand. While both APIs and ORMs serve as intermediaries between an application and a database, ORMs are essentially optional and serve as a layer of abstraction to simplify database processes to programmers and users by presenting them through an object-oriented lens.

4.1.2 Normalization strategies

The design and organization of a relational database is important in determining its potential usefulness – how different informational fields are connected within a database affects what the queries must look like to retrieve them. *Normalization* is the process of

organizing a database to minimize redundancy and dependency while maximizing data integrity and consistency.

Normalization is often measured through a presence (or lack thereof) of certain traits in a database. For example, a very common and beneficial trait for a database to have is when every table has a primary key. This ensures that each of the items in a table can be identified – without such a key, there would be no way to check for duplicate records or distinguish between two rows matching some of the same criteria. A database table with this trait is seen as measurably more normalized than one without it.

There are many of these normalization traits which are commonly organized into "forms" – groups of traits which make up ascending levels of normalization [7]. For example, the lowest level of normalization, First Normal Form (1NF), requires tables to have a primary key (as previously mentioned), as well as other fairly essential traits like not mixing data types within a single attribute and not storing multiple distinct values in a single field. Higher levels of normalization impose further restrictions on the traits database tables can have: Second Normal Form (2NF) requires non-key attributes in a table to depend on the entire primary key – if an attribute depends on only part of a primary key, that could indicate it would be better represented in a separate table; Third Normal Form (3NF) goes a step further and requires those same non-key attributes to *only* depend on the key – if an attribute indirectly depends on the key and directly depends on another attribute, those attributes are also likely to be better represented in their own separate table.

The levels of normalization continue beyond 3NF: further levels include Boyce-Codd Normal Form (BCNF), a slight alteration of 3NF in which every attribute (key or not) must depend on the primary key; and Fourth to Sixth Normal Forms (4NF, 5NF, and 6NF, respectively). Each level solves a different potential problem caused by lack of normalization, though as the levels increase the problems get more complex and less

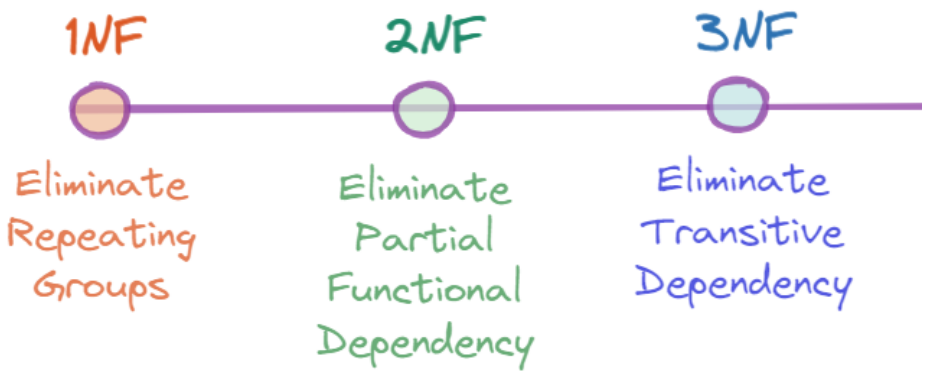


Figure 4.1: A representation of the problems addressed by the first three normal forms

likely to appear in simple or common databases. Thus, the first three normal forms are the ones most commonly described and adhered to.

4.2 Non-relational databases and NoSQL

Relational databases offer a number of advantages, but some contexts do not require all of them and would benefit from a system of database design exchanging unnecessary features for increased resource efficiency and reduced overhead. Non-relational databases exist for this purpose, and usually handle semi-structured or unstructured data that needs more flexibility to store [4]. The term often used to describe non-relational databases is "NoSQL", a broad term used to generally describe the variety of non-relational techniques that can be used to form a database. NoSQL as a concept began to gain traction in the late-2000s and early 2010s among developers looking for alternatives to traditional relational databases [11].

NoSQL as a term represents a variety of system styles, but usually refers to one of the following four:

- **Key-value stores** hold data in simple key-value pairs which are queried and retrieved via the key. The nature of the value can be flexible, and its simplicity reduces computational overhead

- **Column family stores** use tables similarly to relational databases, but preserves the order of the entry rows added, allowing them to orient their queries by column, speeding up data retrieval.

- **Graph stores** store data as a collection of nodes and relationships and are used when the nature or content of the data stored is less important than the web of connections between items

- **Document stores**, which store data directly in formatted documents like JSON or XML. Individual documents are independent, so no foreign keys exist in the system.

In contrast to the ACID model used by relational databases and their managers, NoSQL systems do not prioritize data consistency and integrity above all other considerations. Their alternative to the concept of ACID is BASE, a backronym standing for:

- **BAsic availability** means that the system is allowed to be temporarily inconsistent in order to maintain a constant state of partial availability, meaning that the system is always "basically available"

- **Soft-state** allows for changes in data mid-use and some level of inaccuracy to reduce the amount of consumed resources

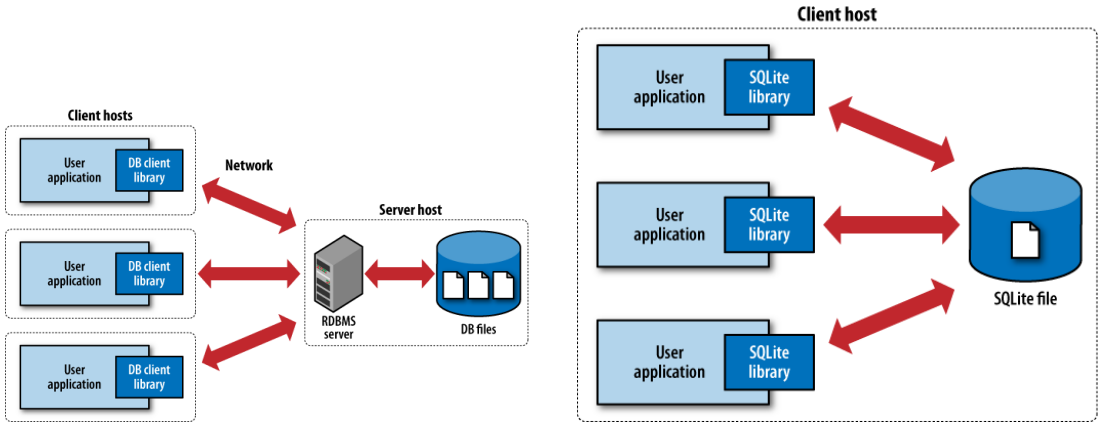
- **Eventual consistency** is the idea that once all outstanding processes are executed, the system will reach a consistent state.

NoSQL is not a single type of system, but rather a variety of them, all connected by serving as alternatives to the traditional relational database model and prioritizing benefits like availability over strict accuracy and consistency.

4.3 One-file and embedded databases

In contrast to the client-server database model, in which a database functions as a separate system from the web server and communicates with it through an API, embedded databases are integrated within an application and thus can be directly queried and controlled by it. Many simply store their information in a single database file. This removes a significant amount of complexity from a system – instead of needing to handle the multitasking and inter-process communication of a client-server database system, embedded databases require little more than the ability to read and write to somewhere in storage.

The most popular embedded database system is SQLite, a software library which uses a relational database model and is designed to be able to be integrated into a wide range of software environments, usually directly into an executable [13].



(a) Client-server database **(b)** Embedded database

Figure 4.2: Comparing the client-server database model with the embedded model

The key drawback to this style of database, however, is that it isn't necessarily as reliable when used outside of a localized context. Embedded databases are not well suited to act as centralized databases, where multiple client machines are able to make concurrent requests. One-file databases are especially risky in this regard, as two concurrent requests operating on the same single file can result in lost transactions, or worse: corrupting the

entire file and breaking the connection between the database and the application. Some one-file database systems like SQLite have built-in functionality to handle this type of concurrency, though they can't match the efficiency of comparable client-server systems [3]; others, like the Python library TinyDB, do not have built-in concurrency handling and must have a queue of requests managed by an external system.

4.4 Evaluating potential database systems

In deciding which database system is best for this application, it is important to define or at least speculate on the context in which it will be used. Currently, we will assume the primary function of the application will be to take a user's one-week listening history as input, scrape each present artist's last.fm page, and determine the average daily listeners for each over the past week of available data. This section will take that assumption and assess the different aspects of a database system discussed in the chapter for their potential utility within the project.

First, we must establish the function a database within the completed web application will fulfill. Like similar Last.fm-based web applications, this one will not concern itself with identifying its users – there would be little point, as the user supplies their own listening history. Even if the app were to access the user's profile directly via the site's API, all the data is stored securely by Last.fm, leaving the purview of this app to simply access it. Thus, the application need only to run each query with the listening history given, taking no further interest in who may be running it.

The next consideration is storing the results of each user's inquiry. Once the scraping of the artists in a user's listening history is completed, certain data points and analytical conclusions will be presented to the user, but the 'raw' data the analytics come from are not displayed. A key factor in choosing whether to make this data available into the future is the timeframe that it covers. Unlike a statistic like an artist's all-time listeners, the

one-week listener average for an artist over a specific seven-day period has increasingly limited value the further you get from that period. While historical data on an artist's daily popularity could potentially be of interest (as the history Last.fm keeps only goes back six months), that would require regular scraping of the artist pages in question, which is beyond the scope of this project. In sum, there's no particularly pertinent need to save the results of a query beyond the initial request-response cycle to the user.

In determining what data *would* be helpful to store in a database, we turn to the most resource- and time-intensive process the web application will have to perform: the scraping. In addition to the time it takes to request a page from a website, the program also has to factor in a short delay to avoid overtaxing the site's servers, which may lead to blacklisting if an IP address used by the app is identified as too much of a strain. This delay, multiplied across a potentially large group of artists in a user's history, can add up fast. The listener history chart we take data from contains a point for each day and is thus updated daily. This means that when the application scrapes an artist page for the daily listeners on each day of the most recently recorded week, that data remains accurate for the rest of the day until the chart is updated.

Because of this, a daily cache of artists' one-week listener averages can be created. For example, if User A submits a listening history with ten artists, all of their pages will be scraped and their one-week listener average will be saved to a database. If User B then submits a listening history with ten artists, the database will be queried for each artist to see if they have a value present from another user on that day having the artist in their history. If three of User B's artists overlap with User A's, those artists' data points can be retrieved from the database much more easily and quickly than their pages can be re-scraped. The non-overlapping artists will have their pages scraped, leading to seventeen artists' one-week listener averages present in the database after Users A and B both submit their histories. Each day, when the charts on Last.fm are updated, the cached

information will go out-of-date, and the database will be emptied and readied for the next day's data.

This setup suggests a basic centralized database is right for the project, stored on the backend of the application. The data stored would take the form of simple key-value pairs, matching an artist's name to their one-week listening average. The standard database system for web apps is the relational model [2], which as discussed above, prioritizes accuracy and consistency in handling its queries. However, because the individual cached data points are not necessary to the function of the application – rather incidental as a helpful way to speed up the scraping process – using relational tools like SQL that prioritize accuracy so heavily might be unnecessary or even overkill, as the impact of losing an entry or two along the way would be relatively minor.

Since the features and flexibility relational database systems offer seem, to an extent, unnecessary, we may be able to turn to NoSQL for an alternative. Key-value NoSQL systems are designed to handle simple data with keys and values without the need for a schema or defined table relationships. The simplicity of our data in particular means it can be potentially implemented in several different types of NoSQL database. A document store like MongoDB, for example, could easily hold our list of key-value pairs, as the JSON-style documents are essentially key-value stores themselves.

The final consideration to make on a potential database system is whether to follow the client-server style or the embedded style. As discussed previously, embedded databases are designed more often to be integrated into a software on the the level of the individual user, not to act as a centralized system that could receive concurrent requests from different clients. Thus, it likely isn't a good idea to use one in our case, as the potential for errors would be worrisome, despite the low potential impact of losing individual queries or even the entire database (as it's refreshed daily). While data loss need not be aggressively minimized, it should not be courted either. The benefits of an embedded

system like SQLite are clear in their conceptual simplicity and low-difficulty queries, but a client-server database is likely the best choice.

CHAPTER 5

Data Visualization with the DOM

Once data about each of the user's listened artists has been collected by the scraper and processed by the backend logic, the data then has to be shown to the user. Creating visualizations directly on the webpage (as opposed to creating a graphic separately) involves interacting heavily with the Document Object Model (DOM) – an interface that structures the layout of webpages. This section will discuss the principles of data visualization that go into representing the data we obtain from the user's listening history, the function and scope of interacting with the DOM, and how the project uses frontend software frameworks, combined with visualization frameworks, to build and display the visualizations.

5.1 Visualization principles

Data visualization is the practice of representing data graphically to communicate relationships to the viewer. By assigning quantitative or categorical variables to visual elements like position and color, visualization techniques can be more informative and accessible than textual or table-based representations can [21]. In the context of web development, data visualization is commonly implemented using browser-based technologies like HTML, CSS, and JavaScript. Creating quality visualizations requires

considering design principles like clear labeling and readability, while also making sure a chart is accurate and fulfills its intended communicative goals.

5.1.1 Creating an informative chart

In the scope of this project, insights about a user's music listening history are derived from processes run on and parallel to the original data. A user's history shows us every track they played, the artist, the album, and the timestamp. Once a list of all the artists in a given history is created, the data on how popular that artist is (in terms of recent daily listeners) can be gathered. With this, we now have the two metrics that would seem to determine how mainstream a user's listening habits are: for every artist in their history, how many times the user listened to them, and how popular they are over the past week. Charting many points of data, each with two simple numerical variables seems to be a clear case in which to use a scatterplot.

The key purpose of a scatterplot is to have a visualization in which the data points are primarily being compared to each other, with the axes providing context about the nature of the distribution as a whole. One drawback of using average daily listeners as a metric for how popular an artist is that it does not lend itself to providing context as to what is a particularly high or low number. For tools that rank artists' popularity in terms of total lifetime listeners the process is simple, as the top spot has been retained by Coldplay for some time. All a tool using that metric has to do is calculate what percentage of Coldplay's total listenership any particular artist has. When using average daily listeners, however, the process is not as simple: which artist is most popular changes from day to day, as does the exact number of daily listeners they have at any given point. Last.fm does have a page ranking the top artists by weekly listenership, but this metric is not quite compatible with our metric, which measures unique listeners on each day, independently. In addition, this page organizes days into discrete weeks, starting on Thursday and concluding the following Friday.

Another potential consideration in creating a chart is the distribution of artists' average daily listeners. While I have observed that the most popular artists at a given time usually hover around 500,000 average daily listeners over the course of a week, the vast majority of artists fail to clear the 100,000 mark, with well-known artists like Green Day and The Beatles tending to orbit 60,000 and 80,000, respectively. This means that even if a user had a fairly even distribution of popular and obscure artists in their listening history, the data would still be clumped toward the bottom. A potential solution for this issue is in changing the scale of the y-axis from linear to logarithmic, as shown in Figure 5.1.

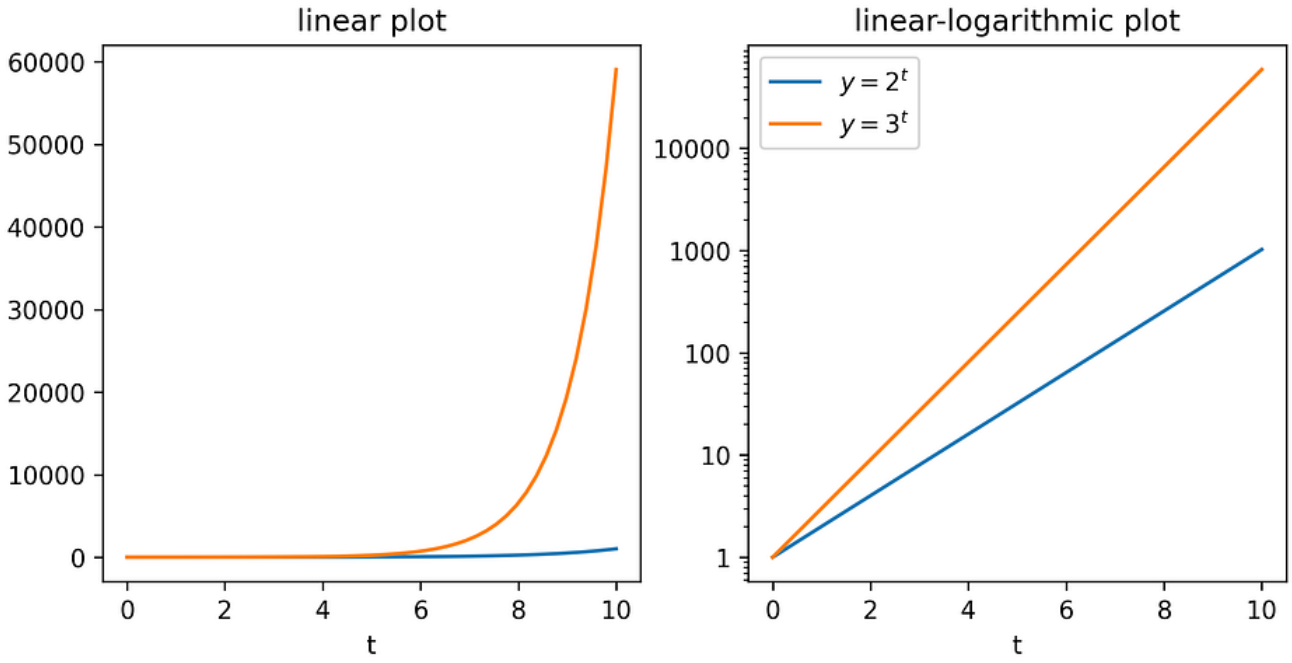


Figure 5.1: An comparison of linear and logarithmic y-axes containing the same data

In the line charts shown, the lines in the logarithmically scaled graph are shaped quite differently than those in its linearly scaled counterpart, engaging in a visual trade-off in an attempt to improve the readability of the chart. While the relationship between the two functions is less intuitive – the lines seem close together while being thousands of units apart – in return the blue line is far easier to reference on the axis, giving a clearer impression of what its value is compared to the linear chart, where its endpoint

inhabits a large space between two ticks. Depending on how pronounced the clumping of the data is toward the bottom of a dataset, a logarithmic scale could be a better option for making each individual data point readable, while sacrificing some of the intuitive understandability of the relationship between many data points.

5.2 The Document Object Model

The DOM is an interface that represents the structure of a webpage as a hierarchical tree of objects (an example is seen below in Figure 5.2) When a browser loads an HTML document, it converts each element (such as paragraph or image blocks) into nodes in this tree. This structured representation allows programming languages like JavaScript to traverse and modify the content and layout of a webpage. Because the DOM is able to be be updated after its initial rendering, it can be coded to respond to user interactions like clicks or keystrokes.

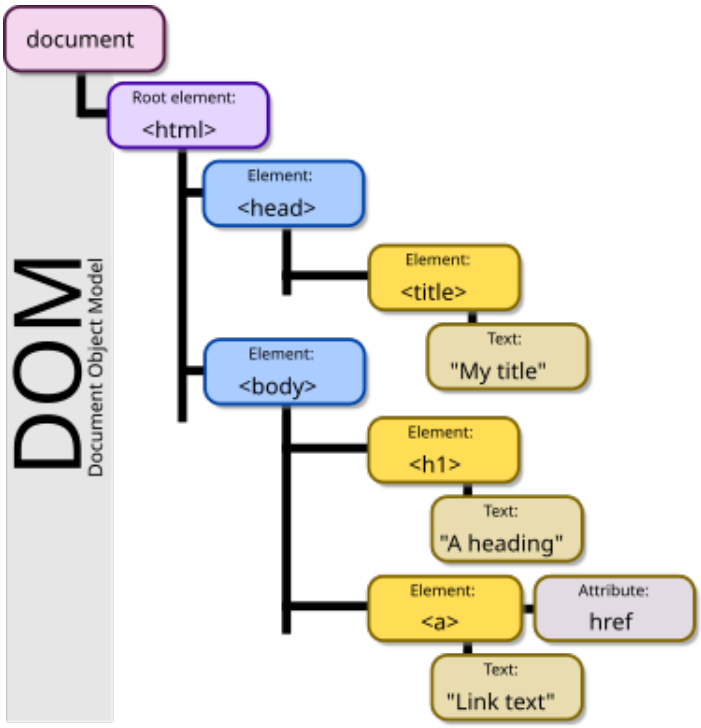


Figure 5.2: An example of a DOM tree

5.3 Vector Graphics in web development

Vector graphics are a method of visual representation often used in web development. While more traditional raster graphics (used in image files like PNGs and JPGs) comprise a set pattern of individual pixels (known as a bitmap), vector graphics store *objects*, which represent individual visual elements within an image. Because of this, they never appear pixelated, no matter the resolution of a screen or how far a user zooms in on them [12].

Another key advantage of vector graphics in a web development context is that they can be rendered directly by the DOM. If the program rendering the graphics have instructions on how to create and modify the necessary objects, it can essentially use the end user's browser to build the image while only needing to store the instructions, saving storage that might otherwise be used for holding much-larger raster image files [16].

As building images manually out of shapes and colors is inherently limiting, a common use case of vector graphics is in creating drawings, diagrams, and visualizations. One notable example of vector usage is on Wikipedia. While many traditional raster images exist on Wikipedia to depict people and objects, vector images are used to represent data using maps and charts. For example, pages concerning the results of elections often feature a map of the territory in question divided into sections, each of which is colored in to correspond with the political party that won the district. An example of such a map is shown in Figure 5.3. In these images, the structured nature of the image allows each element to easily be assigned to an object, which can then each be assigned attributes like colors. The resulting file is then able to be rendered into a PNG of any size for contexts which require a raster image.

5.3.1 The Scalable Vector Graphics format

Scalable Vector Graphics (SVG) is an open-source format for vector graphics. It was initially developed in the late-1990s to capitalize on the emergence of the Extensible

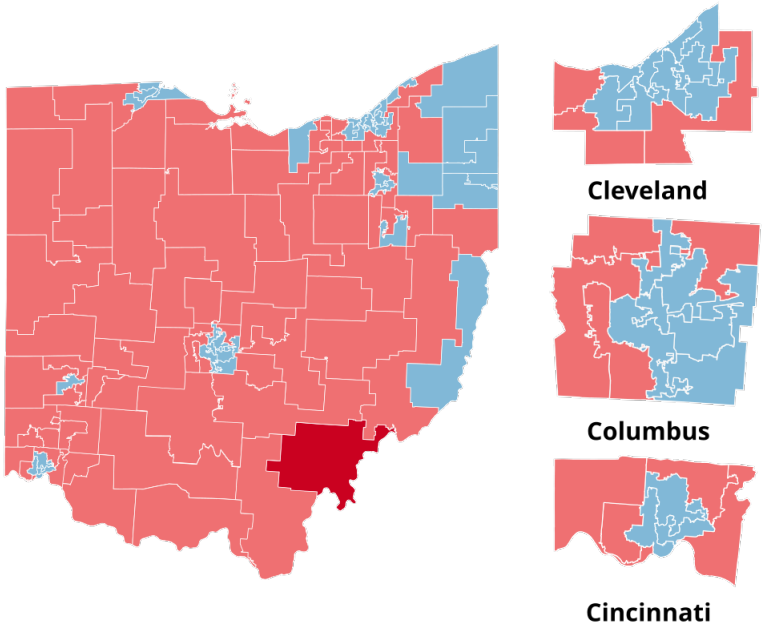


Figure 5.3: An vector graphic of the Ohio 2016 House election results, as used on Wikipedia[1]

Markup Language (XML) format as a method of data encoding. While a new, official version has not been released since 2003, SVG has outlasted several proprietary competitors such as Adobe Flash and is still maintained via newer partial frameworks and draft standards [12]. Using the principles of SVG, we can create visualizations for data that are broadly customizable, don't take up space, and require minimal computing power to generate, all of which are favorable traits in a web development context.

5.4 Using D3 and JavaScript for visualization

D3.js (standing for Data-Driven Documents and often abbreviated to "D3") is an open-source JavaScript library for creating data visualizations within web browsers using previously discussed standards like HTML and SVG. It assigns structured data points to visual elements in the DOM tree and graphically manipulates them with different functions. Because the DOM is manipulable by the user's browser, visualizations can be user-interactive. Rather than having predefined charts, D3 operates at a low level,

requiring each element be specified and designed by the code [16]. While its comparative complexity to other popular data visualization software and technologies gives it a steep learning curve, its extensive flexibility appeals to statisticians, and its compatibility with web standards via JavaScript makes makes it useful to web developers.

5.4.1 Using D3 within a frontend environment

D3 was developed in the early 2010s, before frontend frameworks like React became popular. As a consequence, they can come into conflict in certain ways, most notably in regard to the DOM [16]. React, in particular, creates what it calls the Virtual DOM: a distinct representation of the actual DOM which React creates in JavaScript and updates directly. The Virtual DOM is linked to the "actual" DOM in such a way that it can display small changes to a page and that the actual DOM only needs to be updated when major changes occur [2]. If React is used in tandem with D3, however, this process comes into conflict with certain D3 modules that update the DOM directly [16]. In order to make use of the two, their potential for competing for DOM accessed must be addressed.

There are multiple possible solutions for squaring the two libraries' use of the DOM, each with upsides and drawbacks:

5.4.1.1 Avoiding D3 modules that require DOM access

While many of the important modules in D3 directly modify the DOM, not all of them do. Notably, the modules for generating graph axis scales and specific shapes fall into this category, and can thus be used within a React project without interfering with React's control over the DOM. However, it means that module-enabled functions like binding data to SVG objects, listening for user-generated events, and rendering transitions are now effectively off-limits. While this is very limiting to the actual use of D3 in the project, the good news is that some of this functionality can still be carried out with React. Essentially,

D3 can still perform calculations and generate shapes, while React takes those results and does the legwork of creating the visualization.

The most pertinent responsibility of React with this method is generating the SVG objects to be used in the visualization – since we're making a scatterplot, we need to create the axes the chart comprises and the circles representing each data point. While D3's tools to generate these shapes are unavailable to us, basic SVG objects like shapes can be created with 'vanilla' JavaScript code and can be rendered as such by every major browser.

What we can still use D3 for here is calculating the scale of the axes – how many ticks should each one show, and what numbers they'll display. Since D3 calculates the scales separately from rendering them anyway, all that has to change is to use vanilla JavaScript to render the axes under the specifications D3 generates. This method – of using D3 only for background calculations – loses out on some potential features, but is conceptually the easiest, especially for simple charts.

5.4.1.2 Granting D3 DOM access with hooks

While the previous strategy gave React full control of the DOM by ignoring any D3 modules that directly manipulate it, another strategy sees React effectively delegating full control over a specific portion of the DOM to D3 to create and render the visualization. Due to the disconnect between React and D3's methods, control will be passed from the former to the latter via Hooks – a special type of function used by React to manage the state of the application.

A key building block of React apps are functional components, which are JavaScript functions that accept props – informational parameters formatted like HTML attributes – and return renderable HTML elements in the form of JavaScript XML (JSX). Hooks are a way for React components to access the application's state outside of the normal input-output system of props and return statements, making it easier to link components

to one another and break up separate responsibilities among them. React defines a number of built-in hooks, and allows users to codify their own. For example, the useState hook stores data by initializing a variable and a function to update its value, essentially being a way to create global variables in the application.

To grant D3 access to the DOM without impeding React's management of the virtual DOM, we'll need two built-in hooks: useEffect and useRef. useEffect is used to specify a set of actions to take after a component renders. It takes the form of a function can be set to depend on certain conditions being met before running; if no conditions are specified, however, it simply runs when the component is rendered. useEffect can be used to connect to external systems, calling methods on them and making sure the state of each is in line with the other. This is done in conjunction with useRef, which holds onto a value that isn't needed for rendering, and will not change if the page is refreshed. In our case, useRef would be used to hold onto an entire node of the DOM tree, effectively marking it to be left alone by React while leaving it open for D3 methods to be able to operate on. In this way, D3 and its modules can operate on the section of the DOM linked to by the ref unimpeded.

The downside to this method, however, is that bypassing React's management of D3's allocated DOM node prevents React from optimizing it. React's management of the virtual DOM is central to the framework's functionality, and avoiding it invites potential performance issues. In smaller applications where the chart may be the central focus, it can make sense to bypass the virtual DOM for D3 to access, but in a larger, more complex application, the hit to performance can outweigh the benefit[16]. Thus, the context of the application should be taken into account and weighed against the potential benefit of full D3 access.

5.4.1.3 Choosing a method

In reality, there is more than one way to allocate DOM access to D3 – rather than encapsulating the entire chart within the useRef hook, smaller sections can be allocated, allowing D3 to perform specific tasks, like adding transitions to elements switching from displaying one dataset to another. The overall question, however, remains largely the same: is it worth it for React to allocate portions of the DOM to D3 in order to make use of more features?

In the case of this project, the answer is "no". By choosing a relatively simple type of visualization in the scatterplot and not relying on any transitions or animations of the data, we avoid the need for most D3 modules altogether. Despite building a small-scale application that would likely not be impacted by the performance hit of React surrendering a portion of the DOM, the benefits of doing to are not strong. Given this, relying on vanilla JavaScript for the creation of the SVG objects while D3 handles background calculations like the generation of axes is the right way to go.

CHAPTER 6

Implementation

This section will discuss the application of the topics discussed within the preceding chapters in creating the project's final software component. The end goal of this project is to create a web application that, given a user's music listening history as input, scrapes listener data from Last.fm artist pages and presents to the user data about listener trends of the artists in their history.

6.1 Last.fm as a data source

The nature of the Last.fm database differs from similar services like Spotify's API and necessitates an understanding of how its data is organized and stored to make informed decisions on how to process it and interpret the results.

The most important aspect of Last.fm's data collection and cataloging is that its information is based solely on the literal text of its inputs. Music programs people use to send data to the site send just five pieces of information: artist name, song title, and timestamp – which are all required to create an entry of a played song – plus album title and album artist, which are optional. All of these (with one key exception we will address later) are stripped of their upper- or lower-casing, then interpreted directly by Last.fm and processed into the database as-is, containing no more data than the strings they were sent in as.

This approach differs from other databases like Spotify's collection of artist and listener data or MusicBrainz' publicly editable user-run database because it does not assign a separate ID number to artists, albums, or tracks, instead using the text itself as the primary identifier. The approach introduces certain benefits, drawbacks, and other considerations that will be discussed in the following sections.

6.1.1 Overlapping artists

Because the text of an artist's name is used as a unique identifier, the obvious issue this introduces is that artists that share the same name are indistinguishable from one another. Take Supernova as an example: on one hand it is the name of a Chilean girl group active in the early 2000s who scored a few hit singles in their home country. On another, it is the name of an obscure American 1990s punk band best-known for a novelty single about Chewbacca that was featured on the soundtrack to *Clerks* (1999). Both artists share the same space in the Last.fm system, including the artist page, meaning if you look at the complete list of Supernova's played tracks (shown in Figure 6.1), both bands' songs will be in the list.

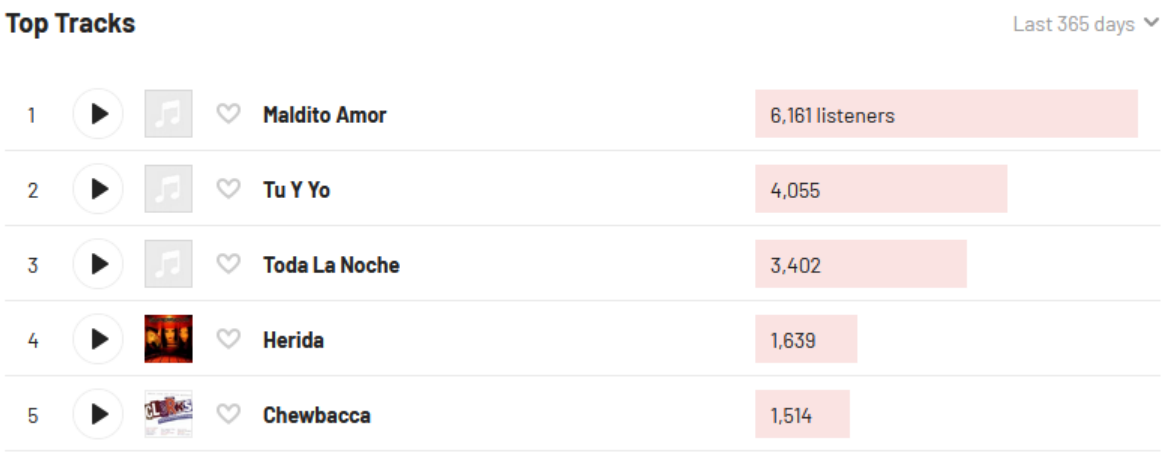


Figure 6.1: The American Supernova's "Chewbacca" is listed among songs by the Chilean Supernova

This presents a problem for our data collection because the Chilean group, while by

no means a hugely popular artist today, still commands a greater listenership on the whole than the American group, despite the latter group's one successful song. This means that using Last.fm data to determine the popularity of the American Supernova is inherently inaccurate, as the data would be presenting the combined listenerships of both, and would indicate that they are far more popular than they actually are.

6.1.2 Misspelled inputs and faulty error correction

Another potential issue with the data Last.fm retains of a user's listening history is that since the inputs come directly from software the user uses, it naturally can contain clerical errors like misspellings. This brings us to the first form of quality-control Last.fm performs on its database: artist redirects. Many artists in Last.fm's database have automatic redirects set up to catch common misspellings and misinputs – for example, 'Brian Transeau' will redirect to his proper stage name 'BT', and 'The Beetles' will redirect to 'The Beatles'. However, this raises a similar problem to the overlapping artists mentioned above: if an artist is actually named "The Beetles" (and multiple do exist), they will be redirected improperly. While this sort of mis-filing rarely happens for high-profile artists, it has been known to happen; in recent years, Last.fm users that are fans of the rapper Lucki have been dismayed to find a redirect in place toward a long-irrelevant Swedish pop duo called Lucky Twice for reasons unclear. In recent years, the site's developers have lost control of redirect functionality entirely, leaving them unable to create new ones or remove those that currently exist.

Redirects aren't quite the same as an artist being filed explicitly under the corrected name. By default, users' submissions of tracks to Last.fm to be recorded are subject to these redirects – if a user scrobbles a song with 'Brian Transeau' listed as the artist, it will automatically be corrected when entered into their history. Users do have the option of disabling this automatic correction for their personal histories, but the greater impact redirects have is on the artist pages themselves.

Redirected artists do have their own pages on the site, but they work slightly differently. If you try to directly navigate to their url as you would a normal artist – by, say, going to https://www.last.fm/music/The+Beetles – you will be rerouted to The Beatles' page. To get around this, you can add "+noredirect" to the url (i.e. https://www.last.fm/music/+noredirect/The+Beetles) to end up at the redirected artist's own page. On it, the page works like normal, having a list of top tracks, a bio, and the six-month listening history chart we take data from. For our purposes, getting to these sites is no trouble: adding "+noredirect" has no effect on links without a redirect, so we can simply add it to the url of every link to be scraped and be able to access their listening data as normal. Unfortunately, while savvy users of the site are likely to have turned off the automatic corrections feature, many others do not, leading to artificially inflated listening figures for the redirectee at the expense of the redirected.

6.1.3 Obtaining and cleaning the data

Last.fm-based web apps that process a specific user's data primarily get access to this data via Last.fm's web API. This service, however, is not universally available. To get access, one must send an application with information on the application you plan to use the API data for, as well as where the site is hosted. Hence, while the site remains in-progress and a decision on a hosting service is yet to be made, API access remains out-of-reach. Potential future use of the official API will be discussed in section 7.2.

Instead of direct API data, we can make use of another Last.fm-based web app that does have API access – specifically, a tool that directly exports listening data. While most tools display or represent a user's data in some way, a few exist that simply export it into a CSV or other specified file type, as the official site has no such functionality. However, some of these tools are geared toward user's exporting their entire history for archival purposes, while we only need data from the past week. After some exploration, the only tool that offers a suitably convenient option to specify a start date for the exported data

is one called Last.fm data export. Because the site is freely accessible, we can base our expectations of user input on what the site outputs as a stopgap method of obtaining a user's listening history until this project can gain access to the official API.

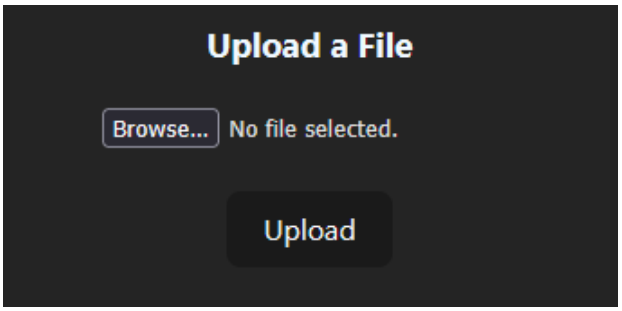


Figure 6.2: The file uploader component as it appears on the site

Within the CSV files the tool outputs, each row is a track, with columns for holding the name, artist, album, and other auxiliary information. To obtain the counts for each artist, we simply need to iterate through the rows, keeping track of how many times each artist has been played. Unfortunately, a key weakness of these export-type tools is that the variety of alphanumeric character used in Last.fm artist names eclipses that of the CSV standard. Because of this, artists with foreign characters in their names (e.g. accents, umlauts, Asian-language symbols) can appear incorrect in the output CSV. Because the CSV files output by the export tool follow a different standard of character encoding, characters outside that standard will be randomized when decoded, creating a garbled text known as mojibake [14]. Currently, the application is configured to accept these CSV files. If an artist's name is distorted by an encoding error, the scraper will fail to find a chart for the artist and they receive a value of -1, but remain in the dataset (artists without any other listeners than he user's in the past six months will encounter the same problem and be treated the same way). A potential future solution will be discussed in section 7.4.

6.2 Building a web app with Flask and React

Construction of the web application was carried out with two popular web development software frameworks to respectively handle the backend and frontend operations: Flask, a Python-based framework that creates the API and manages the database structure through which the site's content is disseminated and stored; and React, a JavaScript framework for designing user interfaces and managing client-side functionality.

6.2.1 Creating a frontend with React

The React frontend is responsible for displaying the application to the end user. It renders the base page, provides the user with graphical elements that accept data to send to the backend, and creates the final visualization with the information it receives back from the backend once the processing has completed.

6.2.1.1 JavaScript XML elements and structure

As discussed previously in subsubsection 5.4.1.2, React applications are primarily constructed with what it calls "functional components" – JavaScript-style functions that return a block of HTML code. The restrictive nature of their inputs and outputs are intended to make such components insular and reusable, easing the process as much as possible to slot an existing component onto an application. React uses an extension to the JavaScript language called JavaScript XML (JSX) which primarily allows it to directly use HTML syntax in its return statements [2]. This allows for greater flexibility than the traditional webpage structure, in which JavaScript is executed within an existing HTML structure. React's method allows the structure of the webpage to be dictated by the client-side code, rather than confine it.

The React code for this software is divided among multiple functional components:

- A top-level component, which establishes the base structure of the page.

- An uploader component, which comprises a place for the user to upload their input file, a container to show the backend's response, and another container to display the completed chart once its data is returned to the frontend.
- Multiple components to handle each phase of assembling and displaying the chart. The structure and function of each will be expounded further below.

Each component has JavaScript code to be executed, and returns HTML elements that are to be graphically rendered. The HTML returns are also able to call and pass props to the other components; the top-level component calls the uploader, which calls the first charting component, which in turn calls the next charting-related component under it.

6.2.1.2 Managing React's state

As mentioned before, React Hooks are how functional components, being largely insular and self-reliant, access the state of the application. In the application, hooks are primarily used in the uploader component, where they are declared to store and manage the communications sent to and received from the backend. State variables are not interchangeable with normally declared JavaScript variables, as any new value they are set to only becomes directly usable after some part of the application updates and re-renders. Thus, state variables are only accessed via hooks when concrete actions are taken with functions, such as a user uploading a file to trigger the function handling the upload, which uses a hook variable to store the name and content of the file. Each component establishes their own hooks, which are limited to the scope of the component, similar to JavaScript variables being limited to the scope of the block they are declared in.

6.2.1.3 Integrating visualizations with D3

A series of nested components is responsible for creating the visualization with the data received from the backend. Once received in the uploader component, the data is passed

as props to a component defining the size and stylings of the chart container, before being passed down farther to a component to create the actual scatterplot along with data about margins and spacing obtained from the container component. The data is then accessed in order to determine the numerical scope of the axes and the attributes of each data point to be added to the plot. The former information is passed twice to components responsible for creating the axes – once for the x-axis, and again for the y – while the latter is passed to a component responsible for creating the circular point that will appear on the plot once for each data point in the dataset. Further details on the visualization process will be discussed later in the chapter.

6.2.2 Creating an API backend with Flask

Flask is responsible for receiving the data sent to it by the frontend and processing it using the application's business logic. It is responsible for extracting from the input files the data on the artist pages to be scraped and the weights to apply to each artist once that data is collected, as well as performing the data analysis to supply the user with insights about their supplied history. Flask also manages the connection to the data store, where scraped data is cached to speed up future processes.

6.2.2.1 Handling user uploads and output data

Once a user designates the file containing their listening history to be uploaded, it is sent to the Flask backend via a POST request over the HTTP protocol. The request is sent to a specific route in the backend (similar to how a browser would access a specific page's URL), which contains the function to handle the incoming file. After checks are made that the file exists in the proper state and format, it is passed to the scraper function, the details of which are expounded in the next section.

Upon the completion of the scraping process, the function returns two things: a textual message for the user describing a few key takeaways from their data, and a JSON file

containing the one week listening average and playcount for each artist to be displayed in the final visualization.

6.3 Processing data with python

Once the backend receives the user's listening history, it then has to process the received information into a list of webpages to scrape for data, find the relevant data within each webpage, and perform the necessary calculations to determine each artist's one week listening average from the scraped data.

6.3.1 Input: reading CSV files

Currently the application is configured to be able to process CSV files from the Last.fm Data Export site – potential future expansion of allowed file types and sources will be discussed in section 7.4. Upon receiving such a file, it is opened and iterated through. Each row represents a track played by the user and contains information about the track's artist and source album, among other things. The software is solely concerned with the artists, however. The first row's artist name is added to a key-value dictionary with the name as a key and a value of 1, representing the count of how many times it has so far appeared. For each subsequent row, the artist value is checked against the dictionary – if the artist is already present, its count is increased; if not, it is added with a count of 1. Once the input file has been completely read through, the dictionary contains the user's play count for each artist in their history.

6.3.1.1 Playcount threshold

Because scraping a site takes so much more time than other Python processes, the number of artists appearing in one's listening history will have a significant effect on the runtime of the program. Regardless of the total amount of tracks they listen to in a given period

of time, some users listen to many different artists, while others only listen to a select few. Additionally, the popularity of mixed-artist playlists on streaming services has made it so, over the course of a week, a large portion of the artists a user listens to are played only a small amount of times – often just once. Since artists played so infrequently naturally will have a very small individual impact on the overall data for an individual user, it makes sense to be able to have a threshold to be able to exclude such entries if the user wants to save time in the scraping process, as how many times each artist was played has no effect on how long scraping that artist's page will take

Currently the program hard-codes a threshold of four plays for an artist to be included in the scraping process. Artists with fewer plays than the threshold will be filtered out between the previous dictionary-making step and the succeeding link-assembly step. Plans for implementing interactive threshold-setting will be discussed in subsection 7.5.1.

6.3.2 Generating a list of target sites

Since Last.fm profile pages take the form https://www.last.fm/music/[artist name], many artists' names can simply be concatenated to the preceding part of the link. However, if the artist's name contains spaces or other select punctuation, those characters need to be replaced in order to form a valid url. Spaces are replaced by pluses, and other characters are replaced with unicode designations like "%3F" for the question mark character (?) based on the ASCII character encoding standard. For example, the name "Chance the Rapper" produces the link "https://www.last.fm/music/Chance+the+Rapper", and the name "+44" produces the link "https://www.last.fm/music/%252B44", using the designation "%252B" for the plus character (+). Currently, the commonly used special characters "/", "?", and "+" are automatically converted.

Links for each artist are created in this manner and stored alongside their corresponding artist name and playcount.

6.3.3 Scraping: BeautifulSoup and Requests

When a list of valid links has been obtained, the software iterates through it, first using the Python library Requests to visit the link and obtain the source code for each profile page and then using the BeautifulSoup library to turn the response into an object containing all the site's code in text form.

Once each page's code has been obtained, the program uses BeautifulSoup's functions to search for tables within it. As the site is structured now, the second table on the page contains the six-month listening history chart. Web scraping projects like this often run the risk of breaking if anything on the site changes, and this one is no exception. If the table we're using changes or disappears, there isn't a way to fix it without going back in and re-coding the program. Thankfully, the site's layout has, as of March 2026, remained the same since July of 2019, so the likelihood of constant fixes being required is low.

Once the table is acquired, each row is searched for its "js-date" and "js-value" keys, which are themselves searched for "datetime" and "data-value" values, respectively. Each row is parsed in this way, and the results are transformed from lists to a Pandas dataframe, which subsection 6.3.4 will detail.

6.3.3.1 Scraping best practices

One of the main roadblocks to web scraping is that many websites do not want to be scraped and will implement design choices to block or hinder potential scrapers. As discussed previously, Last.fm does not disallow the automated access of its artist pages, so it can be assumed that they are not hostile to scrapers in general. Still, it's important to not abuse this potential privilege and remain in the good graces of the administrators.

Existing literature recommends setting a minimum delay between requests and testing the limits of your access through trial and error [17]. A delay of 3 seconds was used during the testing of the software, and no restrictions were detected. This renders any potential methods of disguising the identity of the scraper unnecessary for the time

being, though potential for adding such functionality in the future will be discussed in section 7.3.

6.3.4 Processing: Pandas

The table extracted by BeautifulSoup is then transformed into a Dataframe by Python's Pandas module, which takes from each artist the listener counts from every date and calculates the average over four timeframes: one week, one month (30 days), three months (90 days), and six months (180 days; the maximum stored by the table). These averages are appended to the Dataframe, attached to each artist and their playcount. Currently the software only makes use of the one week timeframe, with no plans to make use of any of the others (the reasoning for this is explained in section 7.6).

To weight the listener averages, first a new column is created in the Dataframe multiplying each artist's playcount by their average listeners over the one week timeframe. The sum of all values in this new "weighted" column is then divided by the sum of all values in the playcount column, yielding a final value of the user's listener average over the timeframe. The significance of this value is: for each song the user listened to in the last week/month/etc., how popular on average was the artist in terms of average daily listeners over the same time period?

For the vast majority of users whose data was computed in testing, this number ranges from 5,000 to 40,000. However on its own the number means very little. After displaying it, the program then performs some additional operations to provide the user with context for their score.

6.3.4.1 Further data analysis

In addition to the basic processing, more context is provided about the artists within the user's dataset. Specifically, which artists had the least and most average daily listeners, and which ones affected the user's final average the most. Determining the most and

least mainstream artists can be solved with a single command each to the Dataframe. Determining the most influential artists requires that, for every artist, the listening average to be recalculated with that artist excluded. Subtracting the overall listener average from this number yields a positive or negative number signifying how much the overall average would have changed if that artist had not been included. For example, if a user listened to Kendrick Lamar – the most popular artist at the time of writing – once and Green Day – a popular artist, but much less so – 30-40 times, Kendrick Lamar would be shown as the most mainstream artist, but Green Day would have a negative value here, meaning if the user had not listened to any Green Day, the number would have been lower than if they had instead not listened to any Kendrick Lamar. This information is passed, in text form, to the frontend to be displayed with the final chart.

6.4 Caching data with TinyDB

During the scraping process, the data store comes into play, being used as a cache for the one-week listening average calculated for each artist. It is implemented centrally using a document database called TinyDB. When the scraper function iterates through the list of queued links, it first checks to see if the artist and their one-week listening average are already recorded in the database. If so, it retrieves that value and skips scraping that artist's page having already gotten the information it would end up calculating with the scraped data. If the artist is not present in the database, once their one-week listener average is calculated, the artist is inserted as a key with the average as its value. Because the day-by-day chart data is updated daily, the Flask application will clear the database if the data within it is from a previous day. In sum, the component is a centralized document database acting as a daily cache of scraped artist data to save the time and processing power of a scrape if the artist appears in another user's history in the same day.

TinyDB, as the name implies, is designed to be lightweight and take up little storage space within an application. It is a library that establishes a one-file database model based on the Python JSON standard, and its data by default is stored in a .json file. As discussed in section 4.3, one-file databases are not ideal for use as centralized databases, as they cannot handle concurrent requests as well as client-server databases, and sometimes are not equipped to handle them at all, which can lead to corrupted data. TinyDB is a database system without its own system of concurrence correction, and there is no procedure currently implemented to prevent potentially corrupting collisions. That said, the risk of such a collision in the current design of the software is low, as the relatively lightweight data being stored does not take long to query or update. In addition, the potential harm caused by any corruption is also low, as the database is refreshed daily anyway and only serves as a convenience rather than an essential function. Still, other potential options without these risks exist, and will be discussed in section 7.7.

6.5 Visualizing the data with D3 and JavaScript

This section will describe the implemented process of creating a visualization of the listenership of the artists in a user's listening history using client-side tools. Vanilla JavaScript and the D3.js module are both used to generate SVG graphics using the data given to the frontend components by the Flask backend.

In this stage, the data will need to be made available in JSON form and accessible via a url, as it will be operated on by D3, a web-based visualization framework. To do this, once the processing is complete and the data is all stored within the Pandas dataframe, it is converted to JSON and saved in a file to the server's local storage (as these JSON files rarely eclipse even a kilobyte of storage space, no mechanism to clear or remove old files is currently necessary at this stage of development). A Flask route exists to retrieve the

contents of a JSON file by its filename, so that name is returned by the function once the file has been generated and saved.

6.5.1 Creating a scatterplot

First, within the uploader component in the frontend, the filename returned from the backend is used to make a request to the url where the JSON data can be accessed. Once that data is received, it is assigned to the hook created for it and passed to the initial chart component as props.

This first component sets up the HTML element the chart container will exist inside. From there, the data is passed to another component responsible for taking those specifications and calling the final set of components below it to build each of the chart's elements in SVG. In this second component, the scales of the axes are generated by finding the minimum and maximum values in both the artists' one-week listener averages and their playcounts. After testing with several sample datasets, it was determined that the Y-axis would be best suited to using a logarithmic scale, as the distribution of artists' listener averages tend to clump up toward the bottom when displayed alongside at least one extremely well-known artist. Figure 6.3 shows a graph of a sample dataset using a linear scale, while Figure 6.4 shows the same data on a logarithmic scale.

While it is more difficult on the logarithmically scaled version to estimate the exact y-value of the data points, it is far more clear for the obscure artists which of them are more often played without totally sacrificing this clarity for the artists with higher listener averages. D3 functions are used to calculate these scales.

The component then, in returning its HTML elements, calls several other components with the data it has calculated or been given:

- The ChartContainer component takes in the size and margin information to create the SVG container the other elements will exist within

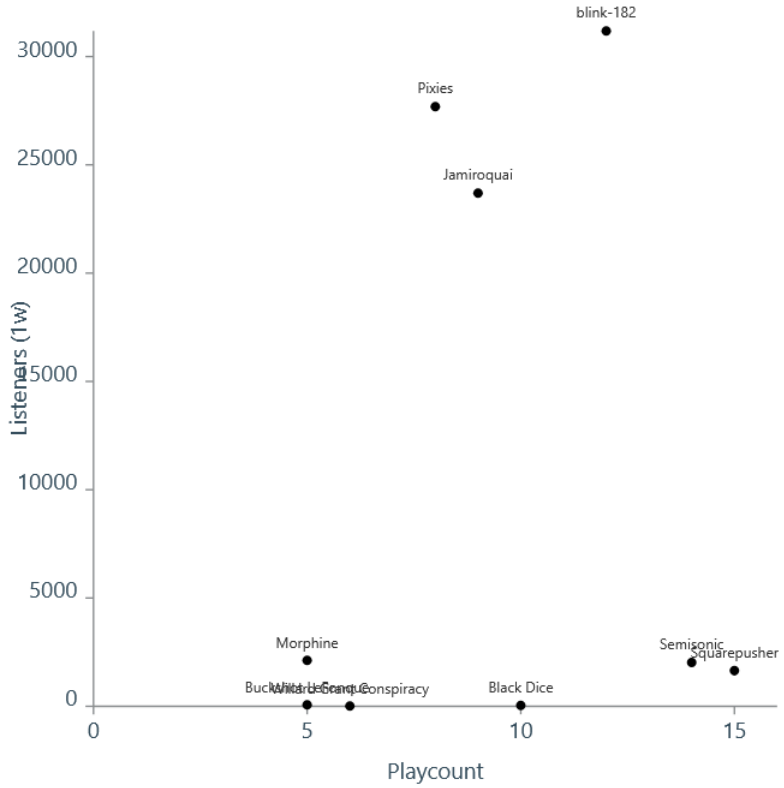


Figure 6.3: An example of several artists graphed on a linear scale

- The Axis component takes the scales calculated by the D3 functions and which side – left or bottom – it will be measuring. It creates tick marks to make each axis readable and applies a text label to each.

- Each data point is mapped onto an instance of the Circle component, each specifying their x- and y-values, along with color and size, which currently all remain uniform. These values are aligned with the axes by using the same x and y scale variables as the axes were given.

- Finally, a text object is attached to each data point, displaying the artist's name. The text is positioned above the point rather than below so that points with low y-values do not have their text intersect the axis.

The result is a scatterplot as exemplified by the previously shown figures, displayed

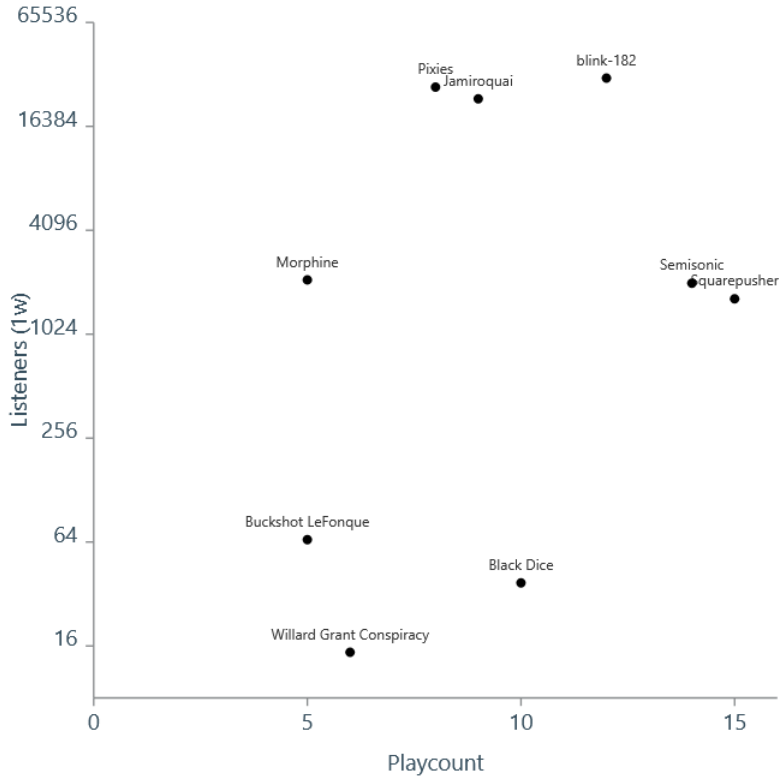


Figure 6.4: An example of several artists graphed on a logarithmic scale

to the user along with the textual information generated by the backend informing the user of their overall one-week listener average, as well as their most impactful artists in bringing that average higher or lower (i.e. which artists would, if removed from the data entirely, cause the average to move furthest in one direction or the other). Functionality to provide specific information about each data point such as their exact x and y coordinates is not currently implemented, and the potential for it will be discussed in section 7.8.

CHAPTER 7

Future Work

This section will discuss the potential for further additions to the application, how they would improve it at fulfilling its purpose, and how these potential improvements tie into the concepts discussed in the main chapters of this paper.

7.1 Deployment

The facet most obviously lacking from the completed software is that it is not deployed. It works in its development environment, but it is not hosted and has no web address – users cannot access it over the internet. This is in equal parts due to complexity and cost: hosting a web application costs money, either from whatever hosting service one uses or through the equipment they maintain to self-host, and the software must be properly configured at a low level to properly integrate with whatever deployment service is being used.

While free options for hosting do exist, they are meant to host small applications, and the ones I have so far explored do not offer sufficient resources to set up a software that has libraries as large and complex as Flask and D3 integrated into it. This leaves paid options which, while more likely to work, require a certain amount of prior research into whether a specific service would be appropriate or viable to deploy the application long-term. Research into this topic is certainly achievable, but was not prioritized in

the development of the project to this point. Deployment would be the primary goal in any continued development of this project and its software so it can fulfill its intended purpose of being a functional and informative tool for music listeners.

7.2 Official API use

Once the application is deployed to the point at which it has an accessible web address, I will be able to apply for access to Last.fm's official developer API. This access would allow the app to obtain a Last.fm user's listening history simply by providing a valid username. While certain limits exist on the extent and amount of requests you can make of the API, the simple gathering of recently-played tracks from one user at a time would not constitute a sizable enough load on the server to run afoul of those limits – many Last.fm-based web apps are based around far more intensive API-calling processes than this.

7.3 Disguising the web scraper

So far in the development of this project, Last.fm has exhibited a tolerance for the scraping the software performs – there has been no indication that the devices or IP addresses used to make the requests have been subject to any sort of blockage or limitation in frequency. While it seems likely that this state of affairs will continue, it is certainly possible that the situation might change. If so, measures would need to be taken to disguise the identity of the software from the site, so it wouldn't be able to as-easily identify the source of the requests.

Normally when a web browser accesses a site, information is sent to the web server about the nature of the user's connection. This includes the IP address of the connecting device, determined by the location of the internet router it uses, along with information

about the browser like its version number. While consistently changing the given IP address would go far in disguising the scraper's identity, it would likely require either a virtual private network (VPN) or a system of connecting to a remote scraping service and be beyond the scope of this project. Instead, changing the HTTP headers sent by the connecting service to resemble that of a normal browser like Chrome of Firefox would be a simpler addition. This, combined with adding a slight random variation to the delay between scrapes, would serve to disguise the activity of the scraper as that of an unusually active user to the server.

7.4 Allowing JSON input

The Last.fm Data Export tool gives the option of exporting a user's data in three possible formats: CSV, XML, and JSON. While each of these data formats have slightly different use cases in general, all three serve the same basic purpose of storing structured data, and are exported from the tool all containing the same information, just organized differently. CSV data was chosen as the initial format for this software to accept due to its compatibility with spreadsheet programs like Excel, which dramatically increase its human-readability compared to the others, easing the early testing process. Unfortunately, the CSV files output by the tool are first converted to an unknown character encoding standard, which garbles and corrupts the text of artists with certain foreign characters in their names.

The other formats evidently do not go through this same encoding process and leave foreign artist names intact – thus, it would be a simple fix to the problem to add functionality to receive and process the differently-structured XML or JSON files as input. Because there is no drawback to any of the three options from the tool, CSV-reading functionality need not be kept, and only one of JSON or XML needs to be potentially readable and accepted by the software.

7.5 Facilitating manual history input

Many avid music listeners do not use Last.fm, and since the only function of accessing a Last.fm user's data is to obtain the counts of how many listens each of their artists had over the past week, the possibility can be raised of a user using an outside method of gathering a listening history to give to this software to access Last.fm's artist popularity data. The simplest way for the software to accept such a history would be for it to allow the user to enter it manually.

Much of this functionality would be implemented on the React frontend. A flexible two-column form could be rendered for the user to enter artists and playcounts into, with a button to add additional rows as needed rather than giving them a predetermined amount. Once the user submits the form, it would be converted to JSON form by the frontend before being sent to the scraper to be decoded. Submitted artist values would then be turned into links and playcounts would be used to weight their listenership figures as normal.

7.5.1 User-adjustable playcount threshold

Similarly, a fronted-based form could also serve to allow the user to set their own minimum plays threshold for an artist to be included in the scraping process. A basic implementation of this idea would be a simple field for a user to specify this threshold directly as they upload the file. The downside to this would be that the user may not be familiar with the distribution of listenerships among the artists in their history and would uninformed as to what a desirable threshold would be. This could be done by first showing the user the distribution after they upload their file but before the scraping begins so they could pick a threshold. This method would be helpful, but involve another round of back-and-forth between the client and server to be implemented.

7.6 Including albums, songs, and lengthier timeframes

As it stands, the scope of the software's function is to gather information about the popularity of artists within a timeframe of one week. Theoretically, it is possible to expand this scope to cover individual albums and songs, as Last.fm generates pages for each that also contain the necessary six-month history graph; and cover time periods of up to six months, the potential time span allowed by the graph. However, as the potentially increased cost of providing this functionality would seem to outweigh the potential benefits, it is not planned to be implemented.

Albums and especially songs are usually far more plentiful in users' listening histories than artists, as you naturally must have at least one of each per artist, and likely many more. The issue is that each of these items has its own page that would need to be scraped just like an artist would, and would thus need to scrape a number of pages many times higher than what would be needed for an artist-based history. While the increased potential load could be managed if tight restrictions were put in place on, say, the maximum number of scrapes allowed by a single user query, it would be simplest to not add the functionality at all and stick to the original scope of the project. This seems reasonable, given that existing Last.fm tools calculating how mainstream a user's taste is also focus only on artists, despite the availability of an overall listenership figure for albums and songs.

7.7 More reliable database model

The software currently uses TinyDB, a database system consisting of a Python library that operates on a simple JSON file containing key-value documents that cache scraped data on artists' listenership until the end of the day. While the simplicity of this system covers all the functionality required of it by the software, its reliance on a single file is not ideal for a centralized system where multiple requests could potentially be made of it at once.

A better way to handle the caching of data would be to implement a database system that uses a client-server model – that way additions or queries by the server would not immediately be carried out on the database, but would instead be queued and handled by the data store as it is able. This could be done either with a relational system like MySQL or a NoSQL system like MongoDB. A NoSQL system would dispense of unnecessary aspect of a relational system not needed for a simple key-value store, but since the number of daily insertions figures to be measured in hundreds, the potential performance benefits would be slight at best.

7.8 Decluttering a crowded scatterplot

Figure 6.4 showed how a logarithmic y-scale was a better choice to prevent the more obscure artists on the chart from overlapping one another. However, it can't fix the problem entirely: if a user generates a scatterplot with dozens of artists or if multiple artists happen to have exceedingly similar x- and y-values, there will be overlapping. One way this problem can be solved is with tooltips.

A tooltip is a small window with information about a data point that generally appears when a user scrolls or taps onto it. Using them raises the possibility of removing the artist names from the scatterplot and housing them solely in the tooltips – this would prevent any overlapping of the text with any of the points or other text. It also would enable the precise y-value of each point to be shown in the tooltip as well, which would make for the weakness inherent to logarithmic axes, that the precise value between two labeled tick marks is difficult to gauge by eye. Tooltips are able to be created in vanilla JavaScript without the use of D3, so adding them would be compatible with the approach described in subsection 5.4.1 of giving React full access to the DOM.

CHAPTER 8

Conclusion

Broadly, the overall goal of this project has been to create a web application using web scraping to access Last.fm data on the size of artists' listenerships in order to inform a user how mainstream their recent music listening has been. While the application is not deployed, and is not accessible through the internet, it functions properly and efficiently as a proof-of-concept, and much of the functionality present in the original version of the software – a simple text-based command line tool – was replicated or expanded on in the web version, giving further indication of the project's potential.

The extent of tools and frameworks available for use in web applications allowed the project to grow in scope. The ability of frontend tools and frameworks like JavaScript and D3 allowed for the visualization of data in such a way that a user gets a much better idea of the shape of their data than could've been provided by the purely text-based conclusions the data analysis could determine. In addition, the ease of connecting the backend to a data store allowed for a method of caching scraped data to save time on redundant future processes. These processes, common and natural among web applications, were able to bring depth and utility to the existing project.

Even more web-based utilities will be of use in future development of the project. In addition to the APIs connecting the components of the software, gaining access to the official API will allow it to communicate with Last.fm in an official and intentional

capacity while also reducing its dependencies by no longer relying on other external tools.

Compared to other Last.fm-based web apps, this one is notably ambitious. Going without official API data and instead relying on scraping – a fairly intensive and time-consuming process to delegate to a backend – is not implemented in any other tools, as far as I know. A conscious choice was made to de-prioritize the research required for finding a suitable hosting service, instead opting to refine the software and improve it as a proof-of-concept. Thus far, that choice feels like a sensible one.

This project was an opportunity for me to introduce myself to foundational concepts of web development on my own terms by designing an application around an area of interest to me. Disappointed with the state of Last.fm web tools that calculate the popularity of artists, I successfully built my own, based on what I wanted to do with the available data. There were many decisions I made along the way, and the background research I performed helped me to both make and understand them. Learning more about databases let me weigh the pros and cons of NoSQL systems; learning about SVG graphics helped me decide whether to create charts on the server-side or in the user's browser. In this way, I was successfully able to connect theory to practice.

Last.fm and the web apps that rely on and augment it have been around for decades now, and will likely remain for years into the future. In that time, more progress can be made on the software and in researching and improving at the skills and knowledge used to create it. Other open-source developers are out there making and maintaining different tools, and their experience can be learned from both impersonally, by studying their software; and personally by reaching out and asking questions. This project serves as the start of a journey to step into this ecosystem, which I hope will continue for me long after it concludes.

APPENDIX A

AI ADDENDUM

The following appendix contains the required documentation of AI use in the writing and software of the project

A.1 WRITING

Introductory paragraphs can be deceptively frustrating to write; I've spent a good amount of time learning about subjects and how to use certain tools, and now I have to describe those ideas succinctly in such a way where a) someone totally unfamiliar with the concept could understand it, and b) it is succinct and written in a formal, academic style. In my view, the difficulty of writing these increases as topics get higher-level – at lower levels, topics can easily be explained in relation to each other by building and expanding on higher ones. At high enough levels, introductory explanations feel like they have to be conjured out of thin air with no supporting context. Describing topics with these constraints is not a strength of mine.

Sometimes out of chance or necessity I can come up with a good introductory paragraph off the top of my head. When I can't, my general strategy (which I expect is fairly common) is to find a description in whatever source I'm using to learn about the subject (often an article or textbook) and use it as a guide. Essentially I use the example as an outline, addressing the same concepts in my own words and with my own

organization if necessary. In the final paper, introductory paragraphs and sentences were created in this way for subjects like APIs, HTTP, and software architecture (as I recall). In situations where I couldn't find a readily available example to use, I used AI to generate one that I could use as a guide instead. The three such situations are described below, in chronological order from the date they took place: introducing the D3 library, the concept of data visualization, and the concept of relational databases. Conversations with the chatbot are notated in sans-serif, and my inputs are bolded. No other use of AI went into the writing of the paper.

A.1.1 D3

in a paragraph, introduce D3.js and its general purpose and functionality

D3.js (Data-Driven Documents) is a powerful open-source JavaScript library used for creating dynamic, interactive data visualizations directly in web browsers. Its general purpose is to help developers bind data to the Document Object Model (DOM) and then transform that data into visual forms—such as charts, graphs, maps, and dashboards—using web standards like HTML, SVG, and CSS rather than proprietary frameworks. D3 provides fine-grained control over every visual element, offering tools for data loading, scaling, layout calculation, animation, and transitions, which makes it highly flexible compared to prebuilt charting libraries. Instead of giving you fixed chart types, D3 supplies low-level building blocks, enabling you to craft custom, responsive, and interactive visualizations tailored to specific data and storytelling needs.

can you tone down the complimentary tone and make it more of an academic description?

D3.js (Data-Driven Documents) is an open-source JavaScript library for producing data visualizations in web browsers by directly manipulating web standards such as HTML, SVG, and CSS. Its primary purpose is to bind structured data to elements in the Document Object Model (DOM) and apply transformations that represent the data visually. The library provides functions for tasks such as data loading, scaling, layout computation, and animated transitions, allowing developers to construct charts, graphs, maps, and other interactive graphics. Rather than supplying predefined chart types, D3.js offers low-level primitives that support flexible, programmatic control over visual encodings and interactions, making it suitable for custom visualization development and research-oriented applications.

For reference, the final paragraph as it appears in chapter 5 is shown below (citations removed):

D3.js (standing for Data-Driven Documents and often abbreviated to "D3") is an open-source JavaScript library for creating data visualizations within web browsers using previously discussed standards like HTML and SVG. It assigns structured data points to visual elements in the DOM tree and graphically manipulates them with different functions. Because the DOM is manipulable

by the user's browser, visualizations can be user-interactive. Rather than having predefined charts, D3 operates at a low level, requiring each element be specified and designed by the code. While its comparative complexity to other popular data visualization software and technologies gives it a steep learning curve, its extensive flexibility appeals to statisticians, and its compatibility with web standards via JavaScript makes makes it useful to web developers.

A.1.2 Data visualization

Can you provide a similarly academic introduction to the subject of data visualization in general, touching on the specifics of web development?

Data visualization is the practice of representing data in graphical or visual forms to facilitate exploration, analysis, and communication of patterns, trends, and relationships. By mapping quantitative or categorical variables to visual properties such as position, color, size, and shape, visualization techniques enable more efficient interpretation than textual or tabular representations alone. Within the context of web development, data visualization is commonly implemented using browser-based technologies including HTML for structure, CSS for presentation, and JavaScript for interactivity and data processing. These standards support the creation of interactive and dynamic visualizations that can respond to user input, update in real time, and integrate with networked data sources. As a result, web-based visualization has become a widely adopted approach for dashboards, analytical tools, and public-facing information systems, balancing accessibility, portability, and technical flexibility. High-quality visualizations additionally require careful consideration of perceptual design principles, appropriate visual encodings, clear labeling, data integrity, and usability, ensuring that representations are accurate, interpretable, and aligned with the intended analytical or communicative goals.

For reference, the final paragraph as it appears in chapter 5 is shown below (citations removed):

Data visualization is the practice of representing data graphically to communicate relationships to the viewer. By assigning quantitative or categorical variables to visual elements like position and color, visualization techniques can be more informative and accessible than textual or table-based representations can. In the context of web development, data visualization is commonly implemented using browser-based technologies like HTML, CSS, and JavaScript. Creating quality visualizations requires considering design principles like clear labeling and readability, while also making sure a chart is accurate and fulfills its intended communicative goals.

A.1.3 Relational databases

in a couple paragraphs, explain the structure and function of relational databases

Relational databases are built on a structured, table-based model in which data is organized into rows and columns. Each table (often called a relation) represents a specific entity—such as customers, orders, or products—and each row represents a unique record of that entity. Columns define the attributes of the records, and every table includes a primary key: a uniquely identifying field that ensures each row can be

referenced unambiguously. Relationships between tables are established through foreign keys, which are columns that link one table's records to those in another. This structure allows data to be stored in a highly organized, normalized form that reduces redundancy and maintains consistency.

Functionally, relational databases use Structured Query Language (SQL) to define, manipulate, and query data. SQL enables users to insert, update, delete, and retrieve data with precision, often combining multiple tables through joins based on their relationships. The relational model supports strong data integrity through constraints, transactions, and indexing, which provide reliable, predictable behavior even in large, multi-user environments. As a result, relational databases are widely used in applications where accuracy, consistency, and the ability to express complex queries are essential, such as finance, inventory management, and enterprise systems.

For reference, the final paragraph as it appears in chapter 4 is shown below (citations removed):

The most common form of database is the relational database. Invented in and popularized since the late 1960s, these databases store information in "relations", which are commonly represented as tables representing a specific entity like customers or products in a retail setting. Within each table, each row represents a unique permutation of that entity (e.g. a different customer), and each column defines a particular attribute of the record (e.g. a customer's name, age, etc.).

Every table includes a primary key: a field (or group of fields) in each table that makes each row uniquely identifiable within it. Relationships between tables are maintained with foreign keys, which associate one table's records to those in another via the first table's primary key. Relational database systems use Structured Query Language (SQL) to operate on stored data. SQL is used to insert, update, delete, and retrieve information in a database. The relational model ensures data integrity, which makes it widely used in contexts where accuracy is essential. Important aspects of relational databases that serve to support data integrity and maintain transaction control are referred to under the acronym ACID

A.2 Software

AI tools were also sometimes used in the development of the software, primarily for troubleshooting issues and generating code. Most of the problems that needed assisted troubleshooting had to do with React and JavaScript. As I hadn't used JavaScript or React in a web context prior to this year, some of the underlying concepts and assumptions were unfamiliar to me and resulted in bugs with unhelpful or nonexistent error codes to diagnose the problem with, necessitating some outside help.

Code was only generated in two contexts: creating and modifying visual React components (such as the file upload window) and adding functionality to the SVG chart

with JavaScript code (such as adding text to the points). As the project went on, my understanding of React and JavaScript improved, and I needed less help with making connections between components.

References

[1] *2016 Ohio House of Representatives election*. en. Page Version ID: 1278224788. Mar. 2025. URL: https://en.wikipedia.org/w/index.php?title=2016_Ohio_House_of_Representatives_election&oldid=1278224788 (visited on 02/05/2026) (page 54).

[2] Olatunde Adedeji. *Full-Stack Flask and React: Learn, code, and deploy powerful web applications with Flask 2 and React 18*. en. Google-Books-ID: xGHWEAAAQBAJ. Packt Publishing Ltd, Oct. 2023. ISBN: 978-1-80323-660-5 (pages 19, 21, 34, 40, 47, 55, 64).

[3] *Appropriate Uses For SQLite*. May 2025. URL: https://sqlite.org/whentouse.html (visited on 03/26/2026) (page 45).

[4] Tim Faulkes. *SQL vs. NoSQL: Which Database is Right for Your Application?* en. URL: https://aerospike.com/blog/sql-vs-nosql/ (visited on 12/01/2025) (page 42).

[5] J. J. Geewax. *API Design Patterns*. en. Google-Books-ID: XWU0EAAAQBAJ. Simon and Schuster, July 2021. ISBN: 978-1-61729-585-0 (page 5).

[6] Mihaela Roxana Ghidersa. *Software Architecture for Web Developers: An introductory guide for developers striving to take the first steps toward software architecture or just looking to grow as professionals*. en. Google-Books-ID: 6E2OEAAAQBAJ. Packt Publishing Ltd, Oct. 2022. ISBN: 978-1-80323-161-7 (pages 23–25, 28).

[7] Qiang Hao and Michail Tsikerdekis. *Grokking Relational Database Design*. en. Google-Books-ID: HKRNEQAAQBAJ. Simon and Schuster, Mar. 2025. ISBN: 978-1-63835-744-5 (page 41).

[8] Michael J. Hernandez. *Database Design for Mere Mortals: A Hands-On Guide to Relational Database Design*. en. Third. Google-Books-ID: MVApxsrcHiAC. Addison-Wesley, July 2021. ISBN: 978-0-13-312227-5 (page 38).

[9] IBM. *What Is a Message Broker? | IBM*. en. Oct. 2021. URL: https://www.ibm.com/think/topics/message-brokers (visited on 03/08/2026) (page 10).

[10] Cam Jackson. *Micro Frontends*. June 2019. URL: https://martinfowler.com/articles/micro-frontends.html (visited on 03/11/2026) (page 30).

[11] Ann Kelly and Dan McCreary. *Making Sense of NoSQL: A guide for managers and the rest of us*. en. Google-Books-ID: 9jozEAAAQBAJ. Simon and Schuster, Sept. 2013. ISBN: 978-1-63835-142-9 (page 42).

[12] Dmitry Kirsanov. *The Book of Inkscape, 2nd Edition: The Definitive Guide to the Graphics Editor*. en. Second. Google-Books-ID: DdMoEAAAQBAJ. No Starch Press, Nov. 2021. ISBN: 978-1-7185-0176-8 (pages 53–54).

[13] Jay A. Kreibich. *Using SQLite: Small. Fast. Reliable. Choose Any Three.* en. Google-Books-ID: v5OYlkt6uKYC. "O'Reilly Media, Inc.", Aug. 2010. ISBN: 978-1-4493-9964-1 (page 44).

[14] Thomas Lamiraud. *Mojibake, when encoding goes wrong*. en. Dec. 2023. URL: https://medium.com/@thomas.lamiraud/mojibake-when-encoding-goes-wrong-0958d0631883 (visited on 03/26/2026) (page 63).

[15] Arnaud Lauret. *The Design of Web APIs, Second Edition*. en. 2nd. Google-Books-ID: CRxoEQAAQBAJ. Simon and Schuster, July 2025. ISBN: 978-1-63343-814-9 (page 7).

[16] Elijah Meeks and Anne-Marie Dufour. *D3.js in Action, Third Edition*. en. Third. Google-Books-ID: S4UMEQAAQBAJ. Simon and Schuster, July 2024. ISBN: 978-1-63343-917-7 (pages 53, 55, 57).

[17] Ryan Mitchell. *Web Scraping with Python: Collecting Data from the Modern Web*. en. Google-Books-ID: 7z_fCQAAQBAJ. "O'Reilly Media, Inc.", June 2015. ISBN: 978-1-4919-1025-2 (pages 14, 17–18, 69).

[18] Yangshun Tay. *In-Depth Overview | Flux*. June 2020. URL: https://web.archive.org/web/20200617175837/https://facebook.github.io/flux/docs/in-depth-overview/ (visited on 11/19/2025) (page 33).

[19] *Using server-sent events - Web APIs | MDN*. en-US. May 2025. URL: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events (visited on 03/08/2026) (page 8).

[20] *What Is The MEAN Stack? Introduction & Examples*. en-us. URL: https://www.mongodb.com/resources/languages/mean-stack (visited on 03/25/2026) (page 20).

[21] Claus O. Wilke. *Fundamentals of Data Visualization: A Primer on Making Informative and Compelling Figures*. en. Google-Books-ID: XmmNDwAAQBAJ. "O'Reilly Media, Inc.", Mar. 2019. ISBN: 978-1-4920-3105-5 (page 49).

[22] William. *Web Application Architecture: The Latest Guide (2025 AI Update)*. en-US. July 2025. URL: https://www.clickittech.com/software-development/web-application-architecture/ (visited on 11/15/2025) (pages 24–25).